UNIVERSITÀ
DEGLI STUDI
FIRENZE

School of Mathematical, Physical and Natural Sciences

M.Sc. Degree in Computer Science
(Resilient and Secure Cyber Physical Systems)

Tesi di Laurea

Un'Ontologia OWL per la Conformità alla Direttiva NIS2 Articolo 21: Framework di Validazione e Ragionamento Automatizzato per le Misure di Gestione del Rischio di Cybersicurezza

An OWL-Based Ontology for NIS2 Directive Article 21 Compliance: Automated Validation and Reasoning Framework for Cybersecurity Risk-Management Measures

Candidate:
Abdul Kader

Supervisor:
Prof. Enrico Francesconi

Anno Accademico 2025/2026

Declaration of Originality

I hereby declare that this study is my own work and has been completed independently. To the best of my knowledge and belief, it contains no material previously published or written by another person, except where proper acknowledgment has been made in the text. This work has not been submitted for any other degree or diploma at this or any other institution.

Signature_____

Abdul Kader

Anno Accademico 2025/2026

C O N T E N T S

Declaration of Originality 2
List of Figures 5
List of Tables 6
Abstract 7
Glossary 8
Abbreviations 9
1. Introduction 10
1.1 Background and Context
1.2 Problem Statement
1.3 Research Objectives
1.4 Research Questions
1.5 Scope and Limitations
1.6 Thesis Structure and Contributions
1.7 Research Significance
1.8 Research Design and Thesis Argument
2. Literature Review 13
2.1 Semantic Web Technologies and Ontologies
2.2 OWL and RDF Standards
2.3 Regulatory Compliance in Cybersecurity
2.4 NIS2 Directive Overview
2.5 Existing Compliance Tools and Approaches
2.6 Ontology-Based Compliance Systems
2.7 Research Gap
3. Theoretical Foundation 19
3.1 Web Ontology Language (OWL 2)
3.2 RDF and RDFS Fundamentals
3.3 Ontology Engineering Principles
3.4 Reasoning in Description Logics
3.5 SPARQL Query Language
3.6 SHACL — Shapes Constraint Language
3.7 SKOS Vocabulary and Semantic Alignment
4. NIS2 Directive Article 21: Requirements Analysis 24
4.1 Article 21 Legal Text and Scope
4.2 Article 21(2) Requirements
4.3 Essential vs. Important Entities
4.4 Required Cybersecurity Measures (a)–(l)
4.5 Compliance Criteria
4.6 Challenges in Manual Compliance Verification
5. Methodology 35
5.1 Ontology Development Process (METHONTOLOGY)
5.2 Requirements Gathering
5.3 Competency Questions Definition
5.4 Class Hierarchy Design
5.5 Property Definition
5.6 Validation Rules Design
5.7 System Architecture
5.8 Technology Stack Selection
6. Ontology Design and Implementation 40
6.1 Ontology Structure and Namespace
6.2 Core Classes

6.3 Object Properties
6.4 Data Properties
6.5 OWL 2 Axioms
6.6 SKOS Annotations and External Alignments
6.7 Competency Questions Validation
6.8 Ontology Validation

7. System Implementation 48
7.1 System Architecture Overview
7.2 Backend Implementation
7.3 Frontend Implementation
7.4 Integration and Testing

8. System Demonstration and Use Cases 53
8.1 Interactive Graph Visualization
8.2 OWL Validation and Reasoning
8.3 SHACL Shapes Validation
8.4 SPARQL Query Interface
8.5 Real-Time Entity Compliance Checking

9. Evaluation and Results 65
9.1 Ontology Completeness
9.2 Validation Accuracy
9.3 Reasoning Performance
9.4 Case Studies
9.5 Comparison with Existing Approaches

10. Discussion 70
10.1 Contributions to the Field
10.2 Limitations
10.3 Future Work

11. Conclusion 74
11.1 Summary of Contributions
11.2 Achievement of Research Objectives
11.3 Future Research Directions

References 78

List of Figures

Figure	Description
Figure 6.1	Turtle/ RDF serialization excerpt
Figure 6.2	OWL equivalent-class definition of Compliant Entity
Figure 6.3	OWL subclass hierarchy for measure families
Figure 6.4	OWL role chain for standard derivation
Figure 8.1	Ontology explorer of representative relations
Figure 8.2	Structural ontology validation results
Figure 8.3	Entity classification and standard derivation
Figure 8.4	shape-specific structural validation results
Figure 8.5	Real-time entity assessment
Figure 8.6	SPARQL competency query and results

List of Tables

Table	Description
Table 1	Glossary of core ontology and compliance terms
Table 2	Abbreviations used throughout the study
Table 3	Article 21 measure categories and standard alignment
Table 4	Ontology technology stack selection
Table 5	SKOS annotation and external alignment summary
Table 6	API endpoint reference

Abstract

The European Union's Network and Information Security Directive 2 (NIS2), formally designated as Directive (EU) 2022/2555, requires essential and important entities to implement cybersecurity risk-management measures under Article 21. Article 21(2) contains ten lettered categories, from (a) to (j). This study operationalizes them as twelve ontology classes by modeling the training component of point (g) separately from basic cyber hygiene and the authentication and secure-communications components of point (j) separately. Compliance verification in practice remains predominantly manual, relying on spreadsheets and informal assessments that are difficult to query, validate, and maintain.

This study presents the design and implementation of a formal OWL 2 DL ontology for representing NIS2 Article 21(2) requirements in machine-processable form. The ontology, published under the persistent URI namespace https://w3id.org/nis2/article21#, connects the twelve operational classes through object properties and logical axioms including equivalentClass, complementOf, allValuesFrom, someValuesFrom, and propertyChainAxiom. The prototype classifies an entity as CompliantEntity when its asserted implementations cover all twelve operational classes, thereby covering all ten legal categories at the selected modeling granularity.

A key contribution is a property chain axiom: usesStandard is defined as the composition of implementsMeasure and basedOnStandard. This permits derivation of entity-to-standard relations from implemented measures. Eight operational classes also carry SKOS exactMatch or closeMatch links to external cybersecurity resources.

The prototype includes three SHACL shapes whose constraints are implemented as shape-specific JavaScript checks, a limited SPARQL SELECT interface supporting five competency questions over basic graph patterns and simple filters, and an on-demand entity checking endpoint that performs an in-memory structural coverage assessment without modifying the ontology. A web-based interface built on Node.js and Express provides access to the demonstrator capabilities, including knowledge graph visualization using vis-network, bounded OWL-style reasoning, shape-specific structural validation, and limited SPARQL querying.

Evaluation demonstrates that all five competency questions are answered correctly by the implemented query evaluator, all three SHACL-informed structural checks pass against compliant instances, and the bounded reasoning engine correctly classifies ExampleCompliantEntity as CompliantEntity and ExampleNonCompliantEntity as NonCompliantEntity. The study establishes an extensible ontological foundation for NIS2 Article 21 compliance automation that can be extended to cover broader NIS2 obligations and integrated with organizational security information systems.

Glossary

Term	Definition
Ontology	A formal, explicit specification of a shared conceptualization of a domain
OWL 2 DL	Web Ontology Language profile guaranteeing decidable reasoning
Reasoner	Software that derives new knowledge from ontology axioms via logical inference
Triple	An RDF statement in the form subject–predicate–object
SPARQL	Standard query language for RDF knowledge graphs
SHACL	W3C standard for validating RDF graphs against structural constraints
SKOS	W3C vocabulary for knowledge organization and concept alignment
NIS2	EU Directive 2022/ 2555 on measures for high common level of cybersecurity
Article 21	NIS2 article requiring risk-management measures; paragraph 2 lists ten lettered categories
Essential Entity	High-criticality organization subject to stricter NIS2 obligations
Important Entity	Significant-criticality organization subject to standard NIS2 obligations
Compliance	Coverage of all ten legal categories through the ontology's twelve operational measure classes
Property Chain	OWL axiom composing two properties: P is derived by composing Q and R
SHACL Shape	A constraint definition targeting a class and specifying expected properties
Namespace	URI prefix identifying an ontology, e.g., https:/ / w3id.org/ nis2/ article21#

Abbreviations

Abbreviation	Full Form
OWL	Web Ontology Language
RDF	Resource Description Framework
RDFS	RDF Schema
SPARQL	SPARQL Protocol and RDF Query Language
SHACL	Shapes Constraint Language
SKOS	Simple Knowledge Organization System
NIS2	Network and Information Security Directive 2
EU	European Union
URI	Uniform Resource Identifier
IRI	Internationalized Resource Identifier
DL	Description Logic
API	Application Programming Interface
UI	User Interface
TTL	Turtle (RDF serialization format)
JSON	JavaScript Object Notation
ISO	International Organization for Standardization
NIST	National Institute of Standards and Technology
ENISA	European Union Agency for Cybersecurity
CSF	Cybersecurity Framework
CQ	Competency Question

Chapter 1: Introduction

1.1 Background and Context

The European Union's Network and Information Security Directive 2 (NIS2), formally designated as Directive (EU) 2022/2555, represents a significant development in EU cybersecurity regulation. Entered into force on 16 January 2023 and superseding the original NIS Directive of 2016, NIS2 substantially broadens the scope of mandatory cybersecurity obligations, extending requirements to a significantly wider range of sectors including energy, transport, banking, health, digital infrastructure, and public administration. Organizations falling within its scope are classified as either essential entities or important entities, each subject to proportionate but rigorous compliance obligations.

Central to the NIS2 framework is Article 21, which mandates that covered entities implement comprehensive cybersecurity risk-management measures. Article 21(2) lists ten categories, from (a) to (j), covering risk analysis, incident handling, business continuity, supply-chain security, secure development, effectiveness assessment, cyber hygiene and training, cryptography, personnel and access governance, authentication, and secure communications. The ontology decomposes two compound provisions into twelve operational classes. Non-compliance can result in administrative fines of up to €10 million or 2% of global annual turnover for essential entities, and €7 million or 1.4% turnover for important entities.

Despite the regulatory clarity of Article 21, compliance verification in practice remains a predominantly manual, resource-intensive process. Organizations typically rely on spreadsheet-based checklists, external audit consultants, and informal self-assessments that lack formal rigor, are not machine-processable, and cannot scale effectively. The gap between the formal legal text and the computational tools available for automated compliance checking represents a compelling research challenge at the intersection of semantic web technologies, knowledge representation, and cybersecurity governance.

Ontology engineering, grounded in the W3C Web Ontology Language (OWL) and the broader Semantic Web stack, provides a well-established approach to formalizing complex domain knowledge for computational processing. Ontologies enable automated reasoning via description logic inference engines, structured querying through SPARQL, and constraint validation through SHACL, providing the technical foundation needed for an automated NIS2 compliance framework.

1.2 Problem Statement

Three fundamental problems motivate this research. First, no formal, machine-processable OWL ontology exists that specifically represents NIS2 Article 21(2) compliance requirements, leaving organizations without a standardized computational basis for automated compliance checking. Second, existing compliance assessment approaches lack the semantic expressiveness to capture logical relationships between cybersecurity measures, the entities subject to compliance obligations, and the risk types these measures address — meaning compliance gaps cannot be inferred automatically. Third,

there is no ontological framework aligning NIS2 requirements with established cybersecurity standards such as ISO/IEC 27001:2022 and the NIST Cybersecurity Framework, making interoperability and cross-reference difficult.

The consequence is that compliance officers and IT security managers must manually interpret legal text, map requirements to controls, and conduct assessments without computational support — a process that is error-prone, slow, and increasingly untenable as the number of regulated entities grows following NIS2's expanded scope.

1.3 Research Objectives

The primary objectives of this study are:

1. Design and implement a formal OWL 2 DL ontology that represents the ten Article 21(2) legal categories through twelve operational measure classes with associated properties and logical axioms.
2. Develop an automated compliance reasoning engine that classifies entities as CompliantEntity or NonCompliantEntity using OWL equivalentClass axioms and property chain inference.
3. Implement shape-specific structural validation corresponding to three SHACL shapes for NIS2 compliance checking.
4. Provide a limited SPARQL SELECT interface supporting five competency questions derived from NIS2 Article 21 requirements over basic graph patterns.
5. Develop an interactive web-based visualization and compliance checking interface accessible to non-technical stakeholders.
6. Establish SKOS semantic alignments linking NIS2 measures to ISO 27001, NIST CSF, CIS Controls, and ENISA guidelines.

1.4 Research Questions

This study addresses the following research questions:

- RQ1: How can NIS2 Article 21(2) cybersecurity risk-management requirements be formally represented in OWL 2 DL with sufficient expressiveness for automated compliance reasoning?
- RQ2: Which OWL 2 axiom patterns are most appropriate for modeling compliance obligations and automatic entity classification?
- RQ3: How can SHACL constraints complement OWL reasoning to provide comprehensive instance validation?
- RQ4: To what extent can competency questions derived from NIS2 Article 21 be answered through SPARQL queries over the implemented ontology?
- RQ5: How can the ontology support on-demand structural compliance checking for arbitrary organizational entities without modifying the ontology structure?

1.5 Scope and Limitations

This study focuses specifically on Article 21(2) of Directive (EU) 2022/2555. The legal text lists ten categories; the ontology uses twelve operational classes by splitting the combined cyber-hygiene/training provision and the combined authentication/secure-communications provision. The scope does not extend to incident reporting obligations (Article 23), management-body obligations (Article 20), or supervisory measures (Chapter VI). The ontology is designed for OWL 2 DL to retain decidable formal semantics.

Limitations include the use of a structural reasoning implementation in Node.js rather than a full-featured OWL 2 reasoner (such as HermiT or Pellet), meaning some advanced OWL DL inferences are approximated. The custom SPARQL engine supports basic graph patterns and FILTER expressions but not UNION, OPTIONAL, or SPARQL 1.1 property paths. The ontology represents requirements as of the 2022/2555 Directive text and may require updates as national transpositions and implementing acts are issued by member states.

1.6 Thesis Structure and Contributions

The remainder of this study is organized as follows. Chapter 2 reviews relevant literature. Chapter 3 provides theoretical foundations. Chapter 4 analyses NIS2 Article 21 requirements. Chapter 5 describes the methodology. Chapter 6 presents ontology design and implementation. Chapter 7 covers system implementation. Chapter 8 demonstrates system functionality. Chapter 9 presents evaluation. Chapter 10 discusses findings. Chapter 11 concludes.

Principal contributions: (1) the first OWL 2 DL ontology for NIS2 Article 21 compliance under the w3id.org persistent namespace; (2) a property chain axiom for inferring security standard usage; (3) a web-based compliance demonstrator integrating bounded OWL-style reasoning, shape-specific structural validation, limited SPARQL querying, and interactive visualization; (4) SKOS alignments to ISO 27001, NIST CSF, CIS Controls, and ENISA; (5) on-demand in-memory entity checking without ontology modification.

1.7 Research Significance

The research addresses an interdisciplinary translation problem: Article 21 is expressed as a legal and organizational obligation, while automated assessment requires explicit concepts, stable relations, and testable conditions. The study therefore contributes both a formal knowledge model and an executable demonstration of that model.

Its academic significance lies in connecting legal interpretation, cybersecurity governance, and Semantic Web engineering within one traceable artifact. The work evaluates not only whether the requirements can be encoded, but also whether the resulting representation can support inference, validation, querying, and explanation.

The framework remains decision support rather than a substitute for legal interpretation, audit evidence, or a determination by a competent authority.

1.8 Research Design and Thesis Argument

The study follows a design-science structure in which a relevant problem is identified, an artifact is constructed, and the artifact is evaluated against requirements derived from the problem domain. The artifact includes the ontology and the supporting application because formal correctness alone does not establish operational usefulness.

The central argument is that OWL reasoning and SHACL validation should remain complementary. OWL captures logical consequences under open-world semantics, while SHACL tests whether a submitted graph satisfies explicit completeness and datatype constraints. SPARQL supplies an independent inspection layer.

The evaluation demonstrates technical feasibility within the declared scope; it does not establish effectiveness across all regulated sectors or organizational environments.

Chapter 2: Literature Review

2.1 Semantic Web Technologies and Ontologies

The Semantic Web, introduced by Berners-Lee et al. (2001), provides a framework for representing and sharing data in a machine-interpretable form through a layered architecture of standards. At its foundation, the Resource Description Framework (RDF) provides a graph-based data model for representing information as subject-predicate-object triples. RDF Schema (RDFS) extends RDF with vocabulary for describing class hierarchies and property domains. The Web Ontology Language (OWL), standardized by the W3C in 2004 and revised as OWL 2 in 2009, adds rich description logic expressiveness to the Semantic Web stack, enabling formal knowledge representation with automated reasoning capabilities.

An ontology, in the context of knowledge engineering, is defined as a formal, explicit specification of a shared conceptualization of a domain (Gruber, 1993). Ontologies provide a structured vocabulary for describing domain entities, their properties, and the relationships between them, expressed in a language with well-defined semantics that enables automated reasoning. The engineering of ontologies for complex regulatory and legal domains has received growing attention as organizations seek to manage compliance obligations computationally.

2.2 OWL and RDF Standards

OWL 2 is defined in terms of Description Logic SROIQ(D), providing a decidable fragment of first-order logic with sufficient expressiveness for knowledge representation tasks. The OWL 2 DL profile, used in this study, guarantees decidability of reasoning tasks including classification, consistency checking, and instance retrieval. Key OWL 2 constructs relevant to compliance modeling include equivalentClass (defining a class as logically equivalent to a complex class expression), someValuesFrom (existential restriction), allValuesFrom (universal restriction), complementOf (class negation), disjointWith (mutual exclusion), and propertyChainAxiom (composing two properties into a single derived property).

The SPARQL 1.1 Query Language (Harris & Seaborne, 2013) provides a standardized mechanism for querying RDF knowledge graphs through SELECT, CONSTRUCT, ASK, and DESCRIBE queries. SPARQL supports basic graph patterns, FILTER expressions, aggregation, and OPTIONAL clauses, enabling sophisticated interrogation of ontological knowledge. The SHACL Shapes Constraint Language (Knublauch & Kontokostas, 2017) complements OWL by providing closed-world structural validation of RDF graphs against user-defined shapes, filling the open-world assumption gap of OWL reasoning for data validation purposes.

2.3 Regulatory Compliance in Cybersecurity

Regulatory compliance in cybersecurity has become increasingly complex with the proliferation of sector-specific and cross-sector regulations. Prior to NIS2, the original NIS Directive (2016/1148/EU) established the first EU-wide cybersecurity framework, but its limited scope and inconsistent national implementation prompted the development of

NIS2. Other significant cybersecurity regulations include the EU Cybersecurity Act (2019/881/EU), the General Data Protection Regulation (GDPR, 2016/679/EU), the Digital Operational Resilience Act (DORA, 2022/2554/EU) for financial entities, and sector-specific frameworks such as PCI DSS for payment card industry and HIPAA for healthcare.

The challenge of compliance verification across these frameworks has driven interest in automated compliance checking approaches. Traditional audit-based methods are costly and provide only point-in-time snapshots of compliance posture. Model-based and ontology-based approaches offer the potential for continuous, automated compliance monitoring that can adapt as both technical configurations and regulatory requirements evolve.

2.4 NIS2 Directive Overview

Directive (EU) 2022/2555 (NIS2) was adopted by the European Parliament on 14 December 2022 and entered into force on 16 January 2023, with a transposition deadline of 17 October 2024. NIS2 expands the scope of the original NIS Directive to cover more sectors and entities, introduces stricter security requirements, and harmonizes supervisory and enforcement frameworks across member states. It distinguishes between essential entities (subject to proactive supervision) and important entities (subject to reactive supervision after incidents).

Article 21 represents the operational core of NIS2, requiring covered entities to take appropriate and proportionate technical, operational, and organizational measures. Article 21(2) enumerates ten legal categories labelled (a) through (j). Point (g) combines basic cyber hygiene with cybersecurity training, while point (j) combines multi-factor or continuous authentication with secured communications and emergency communications. The ontology decomposes these compound provisions into twelve classes to support more specific querying and validation.

2.5 Existing Compliance Tools and Approaches

Existing tools for NIS2 and general cybersecurity compliance span several categories. Commercial GRC (Governance, Risk, and Compliance) platforms such as ServiceNow GRC, RSA Archer, and IBM OpenPages provide workflow-based compliance management but rely on manually maintained control frameworks without formal semantics. The ENISA NIS2 Implementation Guidance provides structured checklists but remains a document-based resource without computational processing capabilities. The NIST National Cybersecurity Center of Excellence has published NIS2 mapping guides relating Article 21 measures to NIST CSF subcategories, but again as static reference documents.

Academic approaches to automated compliance checking have explored model-driven engineering, formal methods, and semantic technologies. Governatori et al. (2006) demonstrated the use of deontic logic for business process compliance. Palmirani et al. (2018) applied OWL to GDPR compliance modeling. Prior academic work has also applied ontologies to ISO 27001 control mapping. However, no existing work specifically addresses NIS2 Article 21 compliance as a formal OWL ontology.

2.6 Ontology-Based Compliance Systems

The application of ontologies to regulatory compliance has been explored in legal

informatics, healthcare, and financial regulation. Hassan (2025) demonstrated a legal ontology for Section 29 of the Australian National Consumer Credit Protection Act, achieving automated compliance checking through OWL reasoning and SPARQL queries — a structurally similar approach to this study applied to credit law rather than cybersecurity. Palmirani and Governatori (2012) proposed a compliance checking pattern using normative ontologies. Francesconi (2020) demonstrated the application of semantic technologies to legal knowledge representation, providing theoretical foundations directly applicable to regulatory compliance ontologies.

These works collectively establish the feasibility and methodological approach of ontology-based compliance checking, while the specific domain of NIS2 Article 21 cybersecurity compliance remains unaddressed in the literature, constituting the research gap this study fills.

2.7 Research Gap

The literature review identifies a clear gap: no formal OWL 2 DL ontology exists for NIS2 Article 21(2) compliance verification. Existing NIS2 compliance tools are document-based and non-computational. Existing OWL compliance ontologies address other regulatory domains. This study addresses this gap by providing the first machine-processable ontological framework for NIS2 Article 21 compliance, combining bounded OWL-style reasoning, shape-specific structural validation, limited SPARQL querying, and SKOS semantic alignment in a prototype compliance demonstrator.

2.8 Critical Synthesis of Prior Work

The literature demonstrates mature component technologies but limited integration at the level of a specific NIS2 obligation. Semantic Web research supplies expressive representation languages, legal-ontology research supplies formalization methods, and cybersecurity standards supply detailed control catalogues.

The unresolved research problem is how these strands can be combined without treating controls, legal requirements, and evidence as interchangeable. The present study responds by assigning separate ontological roles to entities, measure categories, risks, systems, and external standards.

The identified gap concerns domain-specific integration rather than the absence of established Semantic Web technologies.

2.9 Comparison of Compliance Paradigms

Document-centric assessments are accessible but difficult to query and validate consistently. Rule engines automate decisions but often embed interpretation in implementation-specific code. Ontologies make the conceptualization explicit and allow several standards-based mechanisms to operate over a shared graph.

A hybrid ontology approach offers stronger interoperability and explainability because classifications can be traced through named classes and properties. It also permits the same data to support inference, constraint checking, and analytical queries.

No technical paradigm eliminates interpretation; every formal model remains a documented and revisable reading of the legal source.

2.10 Positioning of the Present Study

This study is positioned between legal knowledge representation and applied cybersecurity engineering. It does not attempt a complete ontology of NIS2 and does not replace organizational risk management. It develops a bounded model of Article 21(2) connected to a demonstrable workflow.

The work extends prior compliance-ontology research by addressing a recent EU cybersecurity instrument, combining OWL and SHACL, and exposing the formal model through an interactive application. These choices allow the artifact to be evaluated semantically and operationally.

The bounded scope supports internal coherence, while broader regulatory generalization remains a subject for future research.

2.11 Ontologies Compared with GRC Data Models

Governance, risk, and compliance platforms commonly store controls, requirements, findings, owners, and evidence in relational or document-oriented schemas. These systems can provide mature workflows, access control, reminders, and audit trails. Their weakness is not necessarily lack of automation, but that semantic relationships are often implicit in application configuration and difficult to exchange across products. An ontology makes class and property meaning explicit and assigns stable identifiers that can be reused by multiple tools.

The two approaches are complementary. A production implementation would not replace workflow, evidence repositories, or ticket management with OWL. Instead, the ontology could provide a semantic integration layer: a GRC requirement record could link to an Article 21 class; a technical control could be related to systems and risks; and findings could refer to the exact measure and evidence claim they affect. This division preserves operational capabilities while improving interoperability and traceability.

2.12 Legal Rules, Description Logic, and Constraint Languages

Legal compliance systems use several formal paradigms. Deontic and rule-based languages are suitable for obligations, permissions, exceptions, deadlines, and contrary-to-duty structures. Description logics are strong at taxonomy, class definitions, consistency, and relation-based inference. Constraint languages are strong at testing whether a concrete data submission contains required values. Article 21 includes aspects relevant to all three paradigms.

The present ontology emphasizes classification and structural coverage. It does not formalize temporal obligations, competent-authority procedures, exceptions, or sanctions as executable legal rules. SHACL is used to report missing data, while procedural code calculates a coverage score. A complete legal-compliance architecture could combine an ontology for shared concepts, a rule layer for normative and temporal conditions, SHACL for data contracts, and provenance for evidentiary support.

2.13 Ontology Evaluation Criteria

Ontology evaluation should distinguish correctness, completeness, clarity, coherence,

conciseness, extensibility, and practical usefulness. Correctness asks whether represented statements are defensible in the domain. Completeness asks whether the declared scope is covered. Coherence asks whether axioms permit unintended contradictions. Clarity concerns labels, comments, and understandable modeling choices. Conciseness discourages redundant concepts. Extensibility concerns whether new sectors, evidence, and legal provisions can be added without redesigning the core.

Criterion	Application to This Study	Current Evidence	Remaining Work
Correctness	Legal categories and semantic relations reflect Article 21	Article references, source review, corrected (a)-(j) mapping	Independent legal review
Completeness	All ten legal categories covered through twelve classes	Class and restriction inventory	Sub-requirements and implementing acts
Coherence	Class and property axioms do not create unintended conflict	Parsing, disjointness checks, controlled examples	Complete OWL reasoner
Clarity	Terms and modeling decomposition are understandable	Labels, comments, chapter explanations	User study and multilingual labels
Conciseness	No unnecessary duplicate domain concepts	Compact 526-triple graph	Refactor evidence and assessment concepts carefully
Extensibility	Model can add evidence, sectors, and further NIS2 articles	Stable namespace and modular supporting classes	Versioning and migration policy
Usefulness	Model answers competency questions and supports gap reporting	Queries, endpoints, demonstration cases	Practitioner evaluation

2.14 Semantic Alignment as a Governance Activity

Cross-framework mapping is often presented as a technical lookup, but it is a governance activity. A mapping can vary by standard version, implementation context, scope, and interpretation. For example, an ISO/IEC 27001 control may support one part of an Article 21 category without satisfying the complete legal expectation. The mapping should therefore record who approved it, which editions were compared, what evidence supported it, and whether the relation is exact, close, broader, or narrower.

The ontology currently uses SKOS exactMatch once and closeMatch for the remaining mapped classes. The exactMatch assertion deserves especially careful review because SKOS exact matching is transitive and expresses strong interchangeability within mapping applications. Where legal and control concepts differ in scope or normative force, closeMatch or a custom evidence-backed mapping resource may be safer. Future work should replace bare links with mapping assertions carrying provenance and rationale.

2.15 Explainability in Automated Compliance

Explainability requires more than displaying a final class. A reviewer should be able to identify the rule applied, input facts used, missing facts, source provision, mapping assumptions, and software version. The property-chain result in this study is explainable because the intermediate measure is retained as the reason an entity is associated with a

standard. The missing-measure result is explainable because each absent class is named.

However, explanations can still overstate certainty. A statement such as "NonCompliantEntity" may be interpreted as a legal conclusion even when it is derived from incomplete self-reported data. The interface and report therefore distinguish a modeled coverage result from authoritative compliance determination. A mature system should present confidence, evidence status, assessment date, reviewer, and unresolved conflicts alongside the classification.

2.16 Synthesis of Design Requirements from the Literature

Requirement	Rationale	Implementation in This Study
Stable vocabulary	Supports shared interpretation and reuse	Persistent namespace and labeled OWL resources
Traceable legal source	Makes modeling choices reviewable	article Reference and comments
Formal category definition	Supports repeatable classification	Compliant Entity equivalent class
Closed-world validation	Reports missing submission data	Article21Compliance Shape
Quality constraints	Separates presence from qualitative claims	Measure Quality Shape
Queryability	Tests competency questions and supports analysis	SPARQL SELECT subset
Cross-framework links	Connects legal and technical vocabularies	SKOS mappings and based On Standard
Explainable inference	Allows review of derived results	Missing lists and via-measure provenance
Explicit limitations	Prevents automation from overstating legal certainty	Bounded reasoner and validation discussion

Chapter 3: Theoretical Foundation

3.1 Web Ontology Language (OWL 2)

OWL 2 is the W3C standard for representing rich and complex knowledge about entities, groups of entities, and the relationships between them. It is grounded in Description Logic SROIQ(D), a highly expressive decidable fragment of first-order predicate logic. OWL 2 supports three main profiles: OWL 2 EL (polynomial-time reasoning), OWL 2 QL (query rewriting), and OWL 2 RL (rule-based reasoning). This study uses OWL 2 DL, the full description logic profile, which provides the expressiveness needed for compliance modeling while guaranteeing decidability of key reasoning tasks.

Key OWL 2 DL constructs used in this study include:

• owl:equivalentClass — declares two class expressions logically equivalent, used to define CompliantEntity as exactly the set of entities implementing all 12 measures.

• owl:complementOf — class negation, used to define NonCompliantEntity as the complement of CompliantEntity.

• owl:someValuesFrom — existential restriction, requiring at least one value of a property to belong to a class.

• owl:allValuesFrom — universal restriction, requiring all values of a property to belong to a class.

• owl:disjointWith and owl:AllDisjointClasses — declaring classes mutually exclusive.

• owl:propertyChainAxiom — composing two properties P and Q to derive a third property R: R(x,z) if P(x,y) ‴ Q(y,z).

• owl:qualifiedCardinality — counting restrictions specifying minimum or exact numbers of property values of a given type.

3.2 RDF and RDFS Fundamentals

The Resource Description Framework (RDF) provides the foundation for representing knowledge as directed labeled graphs. RDF models information as triples (subject, predicate, object), where each component is identified by a URI or blank node, and object values may be literals. RDF graphs can be serialized in multiple formats including Turtle (.ttl), RDF/XML (.owl/.rdf), N-Triples, and JSON-LD. This study uses both Turtle (primary development format) and RDF/XML (secondary format for Protégé compatibility) as ontology serializations.

RDF Schema (RDFS) extends RDF with vocabulary for defining class hierarchies (rdfs:subClassOf), property domains and ranges (rdfs:domain, rdfs:range), and annotations (rdfs:label, rdfs:comment). Dublin Core metadata terms (dc:title, dc:creator, dc:description) are used for ontology documentation, following best practices for ontology metadata.

3.3 Ontology Engineering Principles

This study follows the METHONTOLOGY ontology development methodology (Fernández-López et al., 1997), which provides a structured process for ontology construction comprising: (1) specification — defining purpose and competency questions; (2) conceptualization — identifying concepts, properties, and relationships; (3) formalization

— expressing the conceptualization in a formal language; (4) implementation — coding the ontology; and (5) evaluation — verifying the ontology against requirements. Competency questions (CQs), as advocated by Grüninger and Fox (1995), play a central role in guiding ontology design and verifying correctness.

3.4 Reasoning in Description Logics

Description Logic (DL) reasoning derives new knowledge from explicitly stated axioms through formal inference rules. Key reasoning tasks include: (1) satisfiability checking — determining whether a class can have instances; (2) subsumption — determining whether one class is a subclass of another; (3) classification — computing the complete class hierarchy; and (4) instance retrieval — finding all instances of a class. In this study, the reasoning engine applies these principles structurally to classify entities as CompliantEntity or NonCompliantEntity based on their implemented measures, using the equivalentClass axiom as the classification rule.

The property chain axiom owl:propertyChainAxiom [implementsMeasure, basedOnStandard] on the usesStandard property implements a form of role chain reasoning: if entity E implementsMeasure M, and M basedOnStandard S, then E usesStandard S. This inference pattern is directly analogous to DL role chain composition and is a standard OWL 2 DL feature.

3.5 SPARQL Query Language

SPARQL 1.1 (SPARQL Protocol and RDF Query Language) is the W3C standard query language for RDF knowledge graphs. A SPARQL SELECT query specifies a set of triple patterns (the basic graph pattern) that must match triples in the knowledge graph, optionally combined with FILTER expressions for value constraints, GROUP BY for aggregation, and ORDER BY for sorting. SPARQL queries are evaluated against an RDF dataset consisting of a default graph and named graphs.

In this study, SPARQL queries serve two purposes: (1) answering competency questions that verify ontology correctness against requirements, and (2) providing an interactive query interface for compliance analysis. Five competency question queries are implemented, covering: measures addressing critical risks, standards used by compliant entities, incident prevention measures, measures applying to specific systems, and entity-measure compliance mappings.

3.6 SHACL — Shapes Constraint Language

The SHACL Shapes Constraint Language (W3C Recommendation, 2017) provides a language for describing and validating RDF graphs. SHACL defines shapes — node shapes and property shapes — that specify constraints on the structure and values of RDF nodes. Unlike OWL's open-world assumption (the absence of a fact does not imply its negation), SHACL operates under a closed-world assumption, making it suitable for data validation scenarios where completeness is expected.

This study defines three SHACL shapes: (1) Article21ComplianceShape targets nis2:Entity and requires at least one implementsMeasure value in each of the twelve operational classes; (2) MeasureQualityShape targets nis2:RiskManagementMeasure and checks appropriateness, proportionality, state-of-the-art, and description properties; and

(3) RiskLevelShape targets nis2:CybersecurityRisk and checks exactly one riskLevel from the controlled list low, medium, high, or critical.

3.7 SKOS Vocabulary and Semantic Alignment

The Simple Knowledge Organization System (SKOS) is a W3C recommendation providing a standard vocabulary for knowledge organization systems. SKOS concepts (skos:Concept) can be organized into schemes and connected through hierarchical (skos:broader/narrower) and associative (skos:related) relationships. For cross-vocabulary alignment, SKOS provides skos:exactMatch (the two concepts are interchangeable), skos:closeMatch (closely related but not identical), skos:broadMatch, and skos:narrowMatch.

In this ontology, SKOS alignment properties annotate NIS2 measure classes with references to corresponding concepts in external cybersecurity standards: ISO/IEC 27001:2022, the NIST Cybersecurity Framework, CIS Controls v8, and ENISA guidelines. This enables semantic interoperability — tools consuming the ontology can traverse skos:closeMatch links to access related content in aligned standards without duplicating their content.

3.8 Open-World and Closed-World Semantics

OWL follows the open-world assumption: absence of a statement does not establish that the statement is false. This is appropriate for distributed knowledge but problematic when an assessment must identify omitted mandatory information.

SHACL provides the complementary validation perspective by requiring qualified values and reporting violations when expected assertions are absent. Separating these mechanisms avoids forcing data-completeness tests into OWL constructs whose semantics were designed for logical entailment.

A SHACL violation indicates an incomplete or non-conforming graph; it does not by itself prove that the underlying organization lacks the relevant control.

3.9 Semantic Alignment Principles

Concepts from NIS2 and external standards may overlap without being legally identical. Strict OWL equivalence would permit properties and classifications to propagate between concepts and could produce claims stronger than the source material supports.

SKOS close-match relations provide a deliberately weaker alignment. They enable cross-framework navigation and interpretation while preserving the independent normative status of each standard and legal requirement.

Alignment quality depends on documented expert judgment and should be reviewed when either the regulation or the referenced standard changes.

3.10 Entailment, Assertion, and Validation

An RDF graph contains asserted triples, while an entailment regime determines which additional triples follow from them. RDFS entailment can infer types from domain and range declarations and propagate subclass membership. OWL adds class-expression and property semantics. SHACL validation asks a different question: whether focus nodes

conform to shapes in a particular data graph. These operations can produce different but compatible results.

For example, an entity linked through implementsMeasure has an Entity domain and the object has a RiskManagementMeasure range. Under RDFS entailment these types can be inferred even if not asserted. A simple JavaScript type index that reads only explicit rdf:type triples may not reproduce that inference. Conversely, SHACL processors vary in whether and how they apply entailment before validation. Reproducible validation must therefore state the entailment regime.

3.11 Open-World Assumption and Unique Names

OWL does not assume that a missing statement is false, and it does not assume that two different IRIs identify different individuals. The first principle explains why absence of a measure cannot automatically establish legal non-compliance. The second means that distinct names can denote the same resource unless inequality is asserted or follows from other axioms.

The prototype applies application-level assumptions that are common in forms and databases: the submitted measure list is treated as complete for the assessment request, and different measure identifiers are treated as distinct controlled values. These assumptions are practical, but they are not consequences of OWL semantics. The report makes them explicit so that procedural results are not presented as general description-logic entailments.

3.12 RDF Collections and Anonymous Class Expressions

The equivalent-class definition is represented in RDF using blank nodes and rdf:List structures. owl:intersectionOf points to a list whose members are Entity and twelve anonymous owl:Restriction nodes. Each restriction identifies implementsMeasure as owl:onProperty and one operational class as owl:someValuesFrom. The property-chain axiom similarly uses an RDF list containing implementsMeasure followed by basedOnStandard.

These structures are more complex than ordinary named-node triples. A graph visualizer that removes blank nodes can show the named classes and properties but cannot display the full logical expression faithfully. A complete ontology inspection tool must traverse RDF lists and render anonymous restrictions. This is why Protégé or an OWL-aware renderer remains valuable even when the custom browser graph is useful for domain exploration.

3.13 Domain and Range as Inference Axioms

RDFS domain and range declarations are often misread as database constraints. The statement that implementsMeasure has domain Entity means that any subject using the property is inferred to be an Entity. The range RiskManagementMeasure similarly types every object. It does not reject a triple whose subject or object lacks an explicit type; instead, it contributes the missing type under entailment.

Constraint behavior requires SHACL or application validation. The Article21ComplianceShape includes a class condition for implemented values, making non-conforming values reportable. This difference is important in ontology engineering:

domain and range support semantic integration, while shapes express expectations for submitted data.

3.14 SHACL Results and Severity

A SHACL validation report contains an overall conforms value and individual results identifying source shape, focus node, result path, severity, message, and optionally the offending value. Severity allows an organization to distinguish violations from warnings and informational recommendations. The local shapes use Violation for missing operational categories and invalid risk levels, Warning for the three quality booleans, and Info for missing descriptions.

Severity does not automatically determine legal importance. It is part of the validation policy. An organization could promote missing evidence or a false proportionality value to Violation, or define separate shapes for data ingestion and audit readiness. Versioning the shape graph is therefore necessary because changing severity or cardinality changes the operational assessment contract.

3.15 SPARQL Algebra and Prototype Boundaries

A SPARQL query is translated into algebraic operations such as basic graph pattern joins, filter, projection, union, left join, grouping, ordering, and slicing. The prototype executes basic graph patterns by iteratively extending solution mappings with matching N3 quads. It then applies a small set of filter operations and projects requested variables.

This execution strategy is adequate for the included simple SELECT questions but does not implement the complete algebra. Query parsing and query execution must be distinguished: sparqljs can parse constructs that the local evaluator does not process. A conformance claim therefore requires a standards test suite, not merely successful parsing of example queries.

3.16 Formal Mechanism Comparison

Mechanism	Primary Semantics	Use in Thesis	What It Does Not Establish
RDF	Graph assertions	Stores resources and relations	Logical completeness or data quality
RDFS	Subclass, subproperty, domain, range entailment	Defines basic taxonomy and typing relations	Closed-world constraints
OWL 2	Description-logic class and property semantics	Equivalent class, complement, inverses, property chain	That missing assertions are false
SHACL	Validation against shapes	Category, quality, and risk-level checks	General legal validity
SPARQL	Pattern matching and graph query algebra	Competency questions and inspection	Inference unless provided by dataset or engine
Procedural JavaScript	Application-defined algorithms	Coverage score, bounded classification, fixed checks	Standards-complete OWL, SHACL, or SPARQL behavior

Chapter 4: NIS2 Directive Article 21: Requirements Analysis

4.1 Article 21 Legal Text and Scope

Article 21 of Directive (EU) 2022/2555 is titled "Cybersecurity risk-management measures." Article 21(1) requires essential and important entities to "take appropriate and proportionate technical, operational and organisational measures to manage the risks posed to the security of network and information systems which those entities use for the provision of their services or for the carrying out of their activities, and to prevent or minimise the impact of incidents on recipients of their services and on other services." This establishes the general risk-management obligation and introduces the proportionality principle — measures must be calibrated to the entity's size, risk exposure, and the likelihood and severity of incidents.

Article 21(2) states that the measures shall include at least the ten categories labelled (a) through (j). The phrase "at least" establishes a floor rather than a ceiling. The ontology uses twelve classes because two compound legal points are decomposed for analytical purposes: point (g) becomes BasicCyberHygiene and TrainingAwareness, while point (j) becomes MultiFactorAuthentication and SecureCommunications. This decomposition must not be mistaken for two additional legal subparagraphs.

4.2 Article 21(2) Requirements

The ten legal categories of Article 21(2) are as follows:

a. Policies on risk analysis and information system security — requiring documented risk analysis processes and security policies governing information systems.

b. Incident handling — covering detection, analysis, containment, response, and recovery processes for cybersecurity incidents.

c. Business continuity management — including backup management, disaster recovery, and crisis management capabilities.

d. Supply chain security — addressing security in relationships with direct suppliers and service providers, including vulnerability practices.

e. Security in network and information systems acquisition, development and maintenance — covering secure development practices and vulnerability handling.

f. Policies and procedures to assess the effectiveness of cybersecurity risk-management measures — requiring systematic assessment and monitoring.

g. Basic cyber hygiene practices and cybersecurity training.

h. Policies and procedures regarding the use of cryptography and, where appropriate, encryption.

i. Human resources security, access-control policies, and asset management.

j. Use of multi-factor or continuous authentication, secured voice, video and text communications, and secured emergency communication systems where appropriate.

The ontology maps the ten legal categories to twelve OWL classes: RiskAnalysisPolicy (a), IncidentHandling (b), BusinessContinuityManagement (c), SupplyChainSecurity (d), SecureDevelopment (e), EffectivenessAssessment (f), BasicCyberHygiene and TrainingAwareness (both components of g), Encryption (h), HumanResourcesSecurity (i),

and MultiFactorAuthentication plus SecureCommunications (both components of j).

4.3 Essential vs. Important Entities

NIS2 distinguishes two categories of covered entities. Essential entities are those operating in highly critical sectors (Annex I) including energy, transport, banking, financial market infrastructures, health, drinking water, wastewater, digital infrastructure, ICT service management, public administration, and space. Important entities cover additional critical sectors (Annex II) including postal services, waste management, manufacture of critical products, food production, and digital providers.

Both categories must implement Article 21 measures, but essential entities are subject to proactive ex ante supervision, while important entities face ex post supervision triggered by evidence of non-compliance or incidents. The maximum fines also differ: up to €10 million or 2% of global turnover for essential entities, and up to €7 million or 1.4% for important entities. The ontology models this distinction through two subclasses of Entity: EssentialEntity and ImportantEntity.

4.4 Required Cybersecurity Measures (a)-(l)

Each of the twelve Article 21(2) measures addresses specific aspects of the cybersecurity risk management lifecycle. Table 4.1 provides a mapping of each measure to its primary risk domain and the international security standard most closely aligned with it.

Measure	Legal Ref.	Risk Domain	Standard Alignment
Risk Analysis Policy	21(2)(a)	Risk Governance	ISO 27001 A.5
Incident Handling	21(2)(b)	Incident Response	ENISA Guidelines
Business Continuity Management	21(2)(c)	Resilience	ISO 22301
Supply Chain Security	21(2)(d)	Third-Party Risk	ISO 27002 A.5.19
Secure Development	21(2)(e)	Secure SDLC	NIST CSF PR.IP
Effectiveness Assessment	21(2)(f)	Continuous Monitoring	CIS Controls
Basic Cyber Hygiene	21(2)(g)	Hygiene Baseline	CIS Controls v8
Training Awareness	21(2)(g)	Human Risk	ISO 27001 A.7
Human Resources Security	21(2)(i)	Access Control	ISO 27001 A.6
Encryption	21(2)(h)	Data Protection	ISO 27001 A.8.24
Multi Factor Authentication	21(2)(j)	Authentication	NIST SP 800-63
Secure Communications	21(2)(j)	Communications Sec.	ISO 27001 A.8

4.5 Compliance Criteria

Article 21(1) establishes proportionality as the core compliance criterion: measures must be appropriate and proportionate, considering the entity's size, risk exposure, and the likelihood and severity of incidents. Article 21(2) establishes a binary minimum compliance floor: all twelve categories must be addressed. An entity that implements eleven of twelve measures is legally non-compliant. This binary nature of the minimum compliance floor is directly modeled in the ontology through the equivalentClass axiom using owl:intersectionOf with twelve owl:someValuesFrom restrictions, one for each measure

class.

The ontology additionally models three quality dimensions for each measure instance: isAppropriate (boolean), isProportionate (boolean), and isStateOfTheArt (boolean), reflecting the qualitative compliance criteria of Article 21(1). SHACL shapes validate these quality properties through sh:datatype and sh:minCount constraints.

4.6 Challenges in Manual Compliance Verification

Manual compliance verification against Article 21 faces several structural challenges. First, the measures are described in abstract legal language that requires expert interpretation to translate into concrete technical controls — terms like "appropriate," "proportionate," and "state of the art" have no fixed technical meaning. Second, the twelve measures interact and overlap: for example, SupplyChainSecurity and SecureDevelopment both address software security, and MultiFactorAuthentication and HumanResourcesSecurity both address access control. Manual assessments often miss these interdependencies. Third, evidence collection for audits requires gathering documentation across multiple organizational units, systems, and processes — a time-consuming and inconsistent process. Fourth, as Article 21(3) implementing acts are issued by the Commission for specific sectors, compliance requirements will evolve, requiring systematic updates to assessment frameworks. An ontology-based approach addresses all four challenges through formal semantics, logical inference, structured querying, and extensible knowledge representation.

4.7 Normative Interpretation Strategy

The legal analysis identifies the regulated subject, required action, object of the action, and applicable qualifications. Article 21(1) establishes the general duty to adopt appropriate and proportionate measures, while Article 21(2) identifies the minimum domains those measures must cover.

The ontology reflects this distinction by representing the twelve domains as measure classes and the qualitative criteria as properties of measure instances. This preserves the difference between category coverage and contextual adequacy.

The resulting class catalogue is a documented conceptualization, not a verbatim transcription or an authoritative legal interpretation.

4.8 Granularity of the Legal Measures

The twelve categories differ significantly in internal complexity. Encryption is comparatively focused, whereas risk-analysis policy can include governance, responsibilities, review cycles, documentation, and risk-treatment decisions.

The selected granularity treats each lettered provision as one top-level class. Relations to systems, risks, quality properties, and standards provide additional structure without presenting the model as a complete implementation guide.

Evidence-level sub-controls are outside the present scope but can be introduced beneath the stable top-level taxonomy in later versions.

4.9 Legal-to-Technical Traceability

Formalization introduces modeling choices, so each class must remain traceable to the legal source. Stable URIs, labels, comments, and Article 21 references allow a reviewer to move from an inference result back to the relevant provision.

Traceability also supports regulatory maintenance. When implementing acts, national transposition measures, or authoritative guidance alter interpretation, the affected concepts and constraints can be identified systematically.

Annotations support provenance and explanation but do not replace a versioned legal mapping process with expert review.

4.10 Proportionality and Compliance Evidence

NIS2 does not prescribe an identical technical configuration for every entity. Proportionality depends on exposure, size, implementation cost, and the likelihood and severity of incidents. Binary category presence cannot capture this contextual judgment fully.

The model keeps appropriateness, proportionality, and state-of-the-art status explicit and queryable. A production system would additionally connect these assertions to policies, test reports, asset records, risk assessments, and accountable reviewers.

The prototype evaluates represented coverage of the twelve categories; it does not certify the sufficiency or authenticity of organizational evidence.

4.11 Measure-by-Measure Critical Analysis

The following analysis interprets each modeled measure as an assessment domain rather than a checkbox. For every class, it distinguishes the ontology assertion from the organizational evidence required to support that assertion. This distinction is essential because the knowledge graph can organize claims and reveal missing categories, but the credibility of those claims depends on evidence collected through governance, technical testing, and audit procedures.

4.11.1 RiskAnalysisPolicy — Article 21(2)(a)

The measure establishes the governance basis for deciding which assets, threats, vulnerabilities, likelihoods, and consequences require treatment. A policy is not merely a document: it should define ownership, assessment criteria, review frequency, risk acceptance authority, and the relationship between business objectives and security decisions. In the ontology, RiskAnalysisPolicy is modeled as a subclass of OrganizationalMeasure. ExampleRiskAnalysisPolicy is based on ISO27001, addresses DataBreachRisk and InsiderThreatRisk, and applies to CoreBankingSystem and InternalITInfrastructure.

A defensible assessment would examine the following evidence: Approved risk methodology; current risk register; asset and service inventory; treatment plans; management approvals; review records; links between identified risks and implemented controls. The evidence should identify scope, owner, approval status, review date, and any exceptions so that the graph represents more than an unsupported declaration.

Modeling limitation: The ontology confirms category presence and related risks but does

not calculate residual risk, test whether the methodology is consistently applied, or determine whether management acceptance decisions are reasonable. This limitation does not invalidate the class; it defines the boundary between semantic coverage analysis and assurance over real implementation.

4.11.2 IncidentHandling — Article 21(2)(b)

Incident handling requires an operational capability spanning preparation, detection, triage, containment, eradication, recovery, communication, and post-incident learning. Its effectiveness depends on defined roles, escalation thresholds, evidence preservation, contact paths, and integration with reporting and business-continuity procedures. In the ontology, IncidentHandling is modeled as a subclass of OperationalMeasure. ExampleIncidentHandling is aligned with ENISAGuidelines, addresses RansomwareRisk, prevents RansomwareIncident, minimizes ServiceDisruptionIncident, and applies to all three example systems.

A defensible assessment would examine the following evidence: Incident-response policy and playbooks; duty rosters; alert and ticket records; exercise reports; forensic procedures; lessons-learned actions; notification decision logs; recovery validation. The evidence should identify scope, owner, approval status, review date, and any exceptions so that the graph represents more than an unsupported declaration.

Modeling limitation: An asserted IncidentHandling instance cannot demonstrate detection coverage, response speed, staff readiness, or whether reporting deadlines are met under real operational pressure. This limitation does not invalidate the class; it defines the boundary between semantic coverage analysis and assurance over real implementation.

4.11.3 BusinessContinuityManagement — Article 21(2)(c)

Continuity management connects cybersecurity controls to service resilience. It requires priorities derived from business-impact analysis, recovery objectives, tested backup and restoration processes, crisis decision structures, alternative communications, and coordinated return to normal operation. In the ontology, BusinessContinuityManagement is modeled as a subclass of OperationalMeasure. ExampleBusinessContinuity is based on ISO27001, addresses DDoSRisk, minimizes ServiceDisruptionIncident, and applies to CoreBankingSystem.

A defensible assessment would examine the following evidence: Business-impact analysis; recovery-time and recovery-point objectives; backup inventories; immutable or offline backup controls; restoration tests; disaster-recovery exercises; crisis plans and contact lists. The evidence should identify scope, owner, approval status, review date, and any exceptions so that the graph represents more than an unsupported declaration.

Modeling limitation: The current model records that continuity measures exist but does not represent dependencies, recovery sequencing, test recency, backup integrity, or the difference between documented and achievable recovery objectives. This limitation does not invalidate the class; it defines the boundary between semantic coverage analysis and assurance over real implementation.

4.11.4 SupplyChainSecurity — Article 21(2)(d)

Supply-chain security extends the assessment boundary beyond the regulated entity. Relevant practices include supplier criticality assessment, security requirements in contracts, assurance over service providers, vulnerability notification, concentration-risk

analysis, controlled access, and termination or transition arrangements. In the ontology, SupplyChainSecurity is modeled as a subclass of OrganizationalMeasure. ExampleSupplyChainSecurity is based on ISO27002 and addresses SupplyChainCompromiseRisk.

A defensible assessment would examine the following evidence: Supplier inventory and criticality ratings; due-diligence records; contractual clauses; assurance reports; software-component inventories; third-party access reviews; incident-notification clauses; exit plans. The evidence should identify scope, owner, approval status, review date, and any exceptions so that the graph represents more than an unsupported declaration.

Modeling limitation: The ontology links the measure to a risk and standard but does not represent individual suppliers, fourth-party dependencies, contractual exceptions, assurance quality, or changes in supplier posture. This limitation does not invalidate the class; it defines the boundary between semantic coverage analysis and assurance over real implementation.

4.11.5 SecureDevelopment — Article 21(2)(e)

Secure development covers acquisition, development, maintenance, vulnerability handling, and disclosure. It should integrate security requirements, threat modeling, code review, dependency control, testing, release approval, patch management, and coordinated vulnerability disclosure into the system lifecycle. In the ontology, SecureDevelopment is modeled as a subclass of TechnicalMeasure. ExampleSecureDevelopment is based on NISTFramework, addresses SupplyChainCompromiseRisk, prevents UnauthorizedAccessIncident, and applies to WebApplicationPlatform.

A defensible assessment would examine the following evidence: Secure-development standard; threat models; source-review records; SAST and DAST results; dependency and container scans; software bills of materials; penetration tests; patch metrics; vulnerability-disclosure procedure. The evidence should identify scope, owner, approval status, review date, and any exceptions so that the graph represents more than an unsupported declaration.

Modeling limitation: The model does not evaluate development pipelines, severity thresholds, remediation timeliness, test coverage, exploitable findings, or the provenance and integrity of acquired components. This limitation does not invalidate the class; it defines the boundary between semantic coverage analysis and assurance over real implementation.

4.11.6 EffectivenessAssessment — Article 21(2)(f)

Effectiveness assessment closes the governance loop by asking whether measures operate as intended and reduce relevant risk. It includes metrics, control testing, internal audit, independent assurance, vulnerability assessment, exercise outcomes, corrective actions, and management review. In the ontology, EffectivenessAssessment is modeled as a subclass of OrganizationalMeasure. ExampleEffectivenessAssessment is based on ISO27001 and addresses DataBreachRisk.

A defensible assessment would examine the following evidence: Control-test plans and samples; audit reports; key risk and performance indicators; penetration-test reports; exercise evaluations; corrective-action registers; overdue-action escalation; management-review minutes. The evidence should identify scope, owner, approval status, review date,

and any exceptions so that the graph represents more than an unsupported declaration.

Modeling limitation: Boolean quality assertions cannot establish effectiveness. The ontology needs evidence objects, assessment dates, assessors, methods, findings, and remediation status before it can support assurance conclusions. This limitation does not invalidate the class; it defines the boundary between semantic coverage analysis and assurance over real implementation.

4.11.7 BasicCyberHygiene — Article 21(2)(g)

Cyber hygiene provides the baseline practices on which more specialized measures depend. It includes secure configuration, timely updates, account discipline, endpoint protection, network segmentation, phishing resistance, logging, and routine handling of removable media and mobile devices. In the ontology, BasicCyberHygiene is modeled as a subclass of OperationalMeasure. ExampleBasicCyberHygiene is based on CISControls, addresses InsiderThreatRisk, and prevents UnauthorizedAccessIncident.

A defensible assessment would examine the following evidence: Configuration baselines; patch-compliance reports; vulnerability-aging metrics; endpoint coverage; account-review records; phishing simulations; logging coverage; exception registers; remediation tickets. The evidence should identify scope, owner, approval status, review date, and any exceptions so that the graph represents more than an unsupported declaration.

Modeling limitation: The ontology records one example instance and cannot show deployment coverage, configuration drift, unsupported assets, approved exceptions, or whether hygiene practices remain effective over time. This limitation does not invalidate the class; it defines the boundary between semantic coverage analysis and assurance over real implementation.

4.11.8 TrainingAwareness — Article 21(2)(g), training component

Training and awareness should be risk-based, recurring, role-sensitive, and measurable. General awareness is necessary but insufficient for administrators, developers, incident responders, executives, procurement staff, and personnel with access to sensitive or safety-critical systems. In the ontology, TrainingAwareness is modeled as a subclass of OrganizationalMeasure. ExampleTrainingAwareness is based on ENISAGuidelines and addresses InsiderThreatRisk.

A defensible assessment would examine the following evidence: Training curriculum and attendance; role-to-training matrix; completion and assessment results; phishing simulations; induction and refresher records; targeted campaigns; remediation for repeated failures. The evidence should identify scope, owner, approval status, review date, and any exceptions so that the graph represents more than an unsupported declaration.

Modeling limitation: The current representation cannot distinguish attendance from competence, verify role coverage, measure behavioral change, or establish whether training content reflects current threats and organizational responsibilities. This limitation does not invalidate the class; it defines the boundary between semantic coverage analysis and assurance over real implementation.

4.11.9 HumanResourcesSecurity — Article 21(2)(i)

Human-resources security connects the employment lifecycle with access control and

asset accountability. Relevant controls include screening where lawful, confidentiality duties, least privilege, segregation of duties, joiner-mover-leaver processes, privileged-access review, disciplinary processes, and return of assets. In the ontology, HumanResourcesSecurity is modeled as a subclass of OrganizationalMeasure. ExampleHumanResourcesSecurity is based on ISO27002, addresses InsiderThreatRisk, and prevents DataBreachIncident.

A defensible assessment would examine the following evidence: Screening and confidentiality records; role definitions; access approvals; periodic entitlement reviews; privileged-account inventories; transfer and termination tickets; asset-return evidence; segregation-of-duty reviews. The evidence should identify scope, owner, approval status, review date, and any exceptions so that the graph represents more than an unsupported declaration.

Modeling limitation: The ontology does not model people, roles, employment events, access entitlements, legal constraints on screening, or the timeliness and completeness of offboarding actions. This limitation does not invalidate the class; it defines the boundary between semantic coverage analysis and assurance over real implementation.

4.11.10 Encryption — Article 21(2)(h)

Cryptography must be governed as a lifecycle rather than treated as a binary feature. Scope includes approved algorithms and protocols, key generation and storage, certificate management, rotation, revocation, backup, recovery, access to key material, and migration from deprecated mechanisms. In the ontology, Encryption is modeled as a subclass of TechnicalMeasure. ExampleEncryption is based on NISTFramework, addresses DataBreachRisk, prevents DataBreachIncident, and applies to CoreBankingSystem and WebApplicationPlatform.

A defensible assessment would examine the following evidence: Cryptographic policy; data-classification mapping; encryption inventories; TLS and certificate scans; key-management architecture; rotation records; hardware-security-module configuration; exception and migration plans. The evidence should identify scope, owner, approval status, review date, and any exceptions so that the graph represents more than an unsupported declaration.

Modeling limitation: A true value for isStateOfTheArt does not prove algorithm strength, key protection, protocol configuration, coverage of data at rest and in transit, or readiness for cryptographic deprecation. This limitation does not invalidate the class; it defines the boundary between semantic coverage analysis and assurance over real implementation.

4.11.11 MultiFactorAuthentication — Article 21(2)(j), authentication component

MFA reduces reliance on reusable passwords but must be scoped and implemented carefully. Priority areas include privileged access, remote access, cloud administration, sensitive applications, recovery processes, enrollment, factor replacement, bypass controls, and resistance to phishing and push fatigue. In the ontology, MultiFactorAuthentication is modeled as a subclass of TechnicalMeasure. ExampleMultiFactorAuthentication is based on CISControls, addresses DataBreachRisk, prevents UnauthorizedAccessIncident, and applies to InternalITInfrastructure and CoreBankingSystem.

A defensible assessment would examine the following evidence: Authentication policy;

application and account coverage; identity-provider configuration; privileged-access reports; factor-enrollment records; bypass and recovery logs; conditional-access rules; failed and anomalous authentication events. The evidence should identify scope, owner, approval status, review date, and any exceptions so that the graph represents more than an unsupported declaration.

Modeling limitation: The class assertion does not indicate coverage percentage, factor strength, exemptions, legacy systems, enrollment assurance, session controls, or weaknesses in account recovery. This limitation does not invalidate the class; it defines the boundary between semantic coverage analysis and assurance over real implementation.

4.11.12 SecureCommunications — Article 21(2)(j), communications component

Secure communications cover ordinary and emergency channels used for operational coordination. The assessment should consider confidentiality, integrity, availability, identity assurance, metadata exposure, device management, retention, emergency alternatives, and the ability to communicate when primary infrastructure is unavailable. In the ontology, SecureCommunications is modeled as a subclass of TechnicalMeasure. ExampleSecureCommunications is based on ENISAGuidelines and addresses DataBreachRisk.

A defensible assessment would examine the following evidence: Approved communication-channel inventory; encryption and identity settings; mobile-device controls; emergency communication plans; continuity exercises; retention settings; access reviews; supplier assurance. The evidence should identify scope, owner, approval status, review date, and any exceptions so that the graph represents more than an unsupported declaration.

Modeling limitation: The model does not distinguish communication modes, emergency dependencies, endpoint security, metadata protection, interoperability, or availability during a widespread outage. This limitation does not invalidate the class; it defines the boundary between semantic coverage analysis and assurance over real implementation.

4.12 Evidence Expectations Across the Twelve Measures

Measure	Primary Evidence	Indicative Failure Condition
Risk Analysis Policy	Approved risk methodology; current risk register; asset and service inventory; treatment plans; management approvals; review records; links between identified risks and implemented controls.	The ontology confirms category presence and related risks but does not calculate residual risk, test whether the methodology is consistently applied, or determine whether management acceptance decisions are reasonable.
Incident Handling	Incident-response policy and playbooks; duty rosters; alert and ticket records; exercise reports; forensic procedures; lessons-learned actions; notification decision logs; recovery validation.	An asserted Incident Handling instance cannot demonstrate detection coverage, response speed, staff readiness, or whether reporting deadlines are met under real operational pressure.

Measure	Primary Evidence	Indicative Failure Condition
Business Continuity Management	Business-impact analysis; recovery-time and recovery-point objectives; backup inventories; immutable or offline backup controls; restoration tests; disaster-recovery exercises; crisis plans and contact lists.	The current model records that continuity measures exist but does not represent dependencies, recovery sequencing, test recency, backup integrity, or the difference between documented and achievable recovery objectives.
Supply Chain Security	Supplier inventory and criticality ratings; due-diligence records; contractual clauses; assurance reports; software-component inventories; third-party access reviews; incident-notification clauses; exit plans.	The ontology links the measure to a risk and standard but does not represent individual suppliers, fourth-party dependencies, contractual exceptions, assurance quality, or changes in supplier posture.
Secure Development	Secure-development standard; threat models; source-review records; SAST and DAST results; dependency and container scans; software bills of materials; penetration tests; patch metrics; vulnerability-disclosure procedure.	The model does not evaluate development pipelines, severity thresholds, remediation timeliness, test coverage, exploitable findings, or the provenance and integrity of acquired components.
Effectiveness Assessment	Control-test plans and samples; audit reports; key risk and performance indicators; penetration-test reports; exercise evaluations; corrective-action registers; overdue-action escalation; management-review minutes.	Boolean quality assertions cannot establish effectiveness. The ontology needs evidence objects, assessment dates, assessors, methods, findings, and remediation status before it can support assurance conclusions.
Basic Cyber Hygiene	Configuration baselines; patch-compliance reports; vulnerability-aging metrics; endpoint coverage; account-review records; phishing simulations; logging coverage; exception registers; remediation tickets.	The ontology records one example instance and cannot show deployment coverage, configuration drift, unsupported assets, approved exceptions, or whether hygiene practices remain effective over time.
Training Awareness	Training curriculum and attendance; role-to-training matrix; completion and assessment results; phishing simulations; induction and refresher records; targeted campaigns; remediation for repeated failures.	The current representation cannot distinguish attendance from competence, verify role coverage, measure behavioral change, or establish whether training content reflects current threats and organizational responsibilities.
Human Resources Security	Screening and confidentiality records; role definitions; access approvals; periodic entitlement reviews; privileged-account inventories; transfer and termination tickets; asset-return evidence; segregation-of-duty reviews.	The ontology does not model people, roles, employment events, access entitlements, legal constraints on screening, or the timeliness and completeness of offboarding actions.

Measure	Primary Evidence	Indicative Failure Condition
Encryption	Cryptographic policy; data-classification mapping; encryption inventories; TLS and certificate scans; key-management architecture; rotation records; hardware-security-module configuration; exception and migration plans.	A true value for is State Of The Art does not prove algorithm strength, key protection, protocol configuration, coverage of data at rest and in transit, or readiness for cryptographic deprecation.
Multi Factor Authentication	Authentication policy; application and account coverage; identity-provider configuration; privileged-access reports; factor-enrollment records; bypass and recovery logs; conditional-access rules; failed and anomalous authentication events.	The class assertion does not indicate coverage percentage, factor strength, exemptions, legacy systems, enrollment assurance, session controls, or weaknesses in account recovery.
Secure Communications	Approved communication-channel inventory; encryption and identity settings; mobile-device controls; emergency communication plans; continuity exercises; retention settings; access reviews; supplier assurance.	The model does not distinguish communication modes, emergency dependencies, endpoint security, metadata protection, interoperability, or availability during a widespread outage.

Chapter 5: Methodology

5.1 Ontology Development Process (METHONTOLOGY)

The ontology was developed following the METHONTOLOGY methodology (Fernández-López et al., 1997), which provides a structured, iterative process for knowledge engineering projects. METHONTOLOGY consists of six phases: (1) Specification — defining the purpose, scope, and competency questions; (2) Conceptualization — identifying classes, properties, and relationships; (3) Formalization — expressing the conceptual model in a formal language; (4) Implementation — encoding in OWL 2 Turtle and RDF/XML; (5) Evaluation — validating against competency questions, SHACL shapes, and reasoner output; and (6) Documentation — annotating the ontology with Dublin Core metadata and RDFS labels.

The iterative nature of METHONTOLOGY was applied across three development cycles. The first cycle established the core class hierarchy and object properties. The second cycle added OWL 2 axioms (equivalentClass, propertyChain, disjointWith) and SKOS alignments. The third cycle added SHACL shapes, competency question validation, and the NetworkInformationSystem subclass with appliesToSystem property triples.

5.2 Requirements Gathering

Requirements were gathered from three primary sources. First, the legal text of Directive (EU) 2022/2555, specifically Articles 21(1) and 21(2), was analyzed to identify all mandatory compliance requirements. Second, the ENISA NIS2 Implementation Guidance documents were reviewed to understand how the twelve measures translate to technical controls. Third, related ontology-based compliance works were reviewed to identify established ontology design patterns applicable to regulatory compliance modeling. The requirements were formalized as a set of functional requirements (what the ontology must represent) and non-functional requirements (performance, usability, interoperability).

5.3 Competency Questions Definition

Following Grüninger and Fox (1995), five competency questions (CQs) were defined to guide ontology design and serve as acceptance tests for evaluation:

1. CQ1 — Which cybersecurity measures address critical or high-severity risks? (Tests addressesRisk triples and riskLevel properties)
2. CQ2 — Which security standards does a compliant entity use? (Tests property chain inference: usesStandard is derived from implementsMeasure and basedOnStandard)
3. CQ3 — Which measures prevent specific types of incidents? (Tests preventsIncident object property triples)
4. CQ4 — Which measures apply to a specific network and information system? (Tests appliesToSystem triples for CoreBankingSystem)
5. CQ5 — For each entity, what measures has it implemented? (Tests implementsMeasure property across entity instances)

Each CQ was translated into a SPARQL SELECT query and verified against the ontology. Correct answers for all five CQs were determined from the ontology content before query

execution, enabling objective evaluation.

5.4 Class Hierarchy Design

The class hierarchy was designed top-down from the regulatory analysis. The principal classes are Entity, RiskManagementMeasure, NetworkInformationSystem, CybersecurityRisk, SecurityIncident, and SecurityStandard. EssentialEntity and ImportantEntity are disjoint subclasses of Entity. TechnicalMeasure, OperationalMeasure, and OrganizationalMeasure are disjoint subclasses of RiskManagementMeasure, and the twelve operational classes are assigned beneath one of those categories. CompliantEntity is defined as an Entity satisfying the twelve class-specific implementation restrictions.

5.5 Property Definition

Object properties capture the core relationships required for the prototype: implementsMeasure (Entity to RiskManagementMeasure), isImplementedBy as its inverse, addressesRisk, preventsIncident, minimizesImpact, basedOnStandard, appliesToSystem, transitive hasSubMeasure, and derived usesStandard. The five datatype properties are riskLevel, isAppropriate, isProportionate, isStateOfTheArt, and measureDescription.

5.6 Validation Rules Design

Three layers of validation were designed. Layer 1 (OWL Reasoning): the equivalentClass axiom defines CompliantEntity through logical necessary and sufficient conditions, enabling automatic entity classification. Layer 2 (SHACL): three shapes provide structural validation. Article21ComplianceShape uses sh:qualifiedMinCount 1 on each of the 12 measure classes to verify an entity implements at least one instance of each. MeasureQualityShape verifies measure instances carry quality boolean properties. RiskLevelShape verifies a valid riskLevel string from the permitted enumeration {low, medium, high, critical}. Layer 3 (Competency Questions): five SPARQL queries provide semantic validation of ontology content against domain requirements.

5.7 System Architecture

The system follows a three-tier architecture. The data tier comprises the Turtle and RDF/XML ontology serializations plus a SHACL shapes file. At runtime the server selects the RDF/XML file when present, otherwise the Turtle file, and caches the parsed quads in an N3 store. The application tier exposes ontology visualization data, structural validation, bounded reasoning, shape-specific validation, limited SPARQL SELECT execution, and real-time entity checking. The presentation tier is a single-page web application served from the public directory.

5.8 Technology Stack Selection

Technology selection was guided by three criteria: alignment with W3C OWL 2 ontology representation, lightweight prototype deployment (no external database or reasoner dependency), and open-source availability. The selected stack is:

Component	Technology	Justification

Component	Technology	Justification
Ontology Language	OWL 2 DL (Turtle + RDF/ XML)	W3C standard, decidable reasoning
RDF Parsing	N3.js v1.x	Pure JavaScript, Turtle/ RDF/ XML support
SPARQL Engine	sparqljs + N3.Store	Lightweight; supports basic SELECT patterns
HTTP Server	Node.js + Express	Lightweight REST API
Graph Visualization	vis-network v9	Interactive network graphs
Shape Validation	Custom JavaScript checks	Shape-specific; not a general SHACL engine
Frontend	HTML5/ CSS3/ JavaScript	No framework dependencies

5.9 Iterative Research Workflow

The development process was iterative rather than strictly sequential. Competency questions informed the conceptual model, implementation exposed ambiguous property definitions, and validation outcomes prompted revisions to instances and constraints.

Reproducibility is supported by keeping the ontology serializations, SHACL shapes, application code, and report generator in one project. A reviewer can inspect the formal artifact, execute the example assessments, and regenerate the documentation.

Reproducibility of the software does not remove the need to document legal interpretation and environmental assumptions.

5.10 Competency-Question Derivation

The competency questions were selected to cover distinct information needs: risk coverage, standards alignment, incident prevention, system applicability, and entity-to-measure implementation. Together they exercise the principal classes and object properties.

A question is considered satisfied when its concepts can be represented, a query can be expressed over the vocabulary, and the returned result can be verified against asserted or inferred facts.

The five questions test the declared scope but are not an exhaustive benchmark for all compliance information needs.

5.11 Multi-Layer Validation Design

Validation operates at syntactic, structural, logical, and functional levels. Parsing confirms legal RDF syntax; structural checks confirm declarations and values; reasoning tests classification and property chains; functional tests confirm that the application exposes the expected outcomes.

Positive, negative, and partial entities are necessary because a fully compliant example alone could conceal an inability to detect gaps. The designed cases verify both successful classification and specific missing-measure reports.

Synthetic cases improve control over expected outcomes but provide less external validity than field data from regulated organizations.

5.12 Threats to Validity

Construct validity is limited by representing compliance through asserted ontology data rather than direct operational evidence. Internal validity is constrained by the custom reasoner, and external validity is constrained by the use of illustrative entities.

The study mitigates these threats through explicit scoping, predefined expected results, standards-based representations, and transparent separation between ontology semantics and application-level calculations.

Independent legal review, practitioner studies, and larger real-world datasets are required before broader claims can be justified.

5.13 Operationalization of Legal Concepts

Operationalization converts abstract legal concepts into observable data requirements. For category coverage, the observable claim is an implementsMeasure relation to an individual typed as one of the twelve classes. For appropriateness, proportionality, and state of the art, the prototype uses boolean properties. For risk and system scope, it uses addressesRisk and appliesToSystem. For external context, it uses basedOnStandard and SKOS mappings.

This operationalization is deliberately modest. Boolean values simplify demonstration but compress judgments that normally require criteria, evidence, reviewer identity, date, and rationale. A more mature model would represent an Assessment class connecting a measure, assessor, evidence set, method, result, confidence, validity period, and findings. The present properties can then become derived summaries rather than unsupported primary facts.

5.14 Data Collection Protocol for a Real Assessment

A real deployment would begin by defining the assessment boundary: legal entity, regulated services, locations, systems, suppliers, and responsible management bodies. Data collection would then combine document review, interviews, configuration evidence, technical tests, and observation. Each evidence item should receive a stable identifier and metadata describing source, owner, collection date, sensitivity, retention, and the measure claims it supports.

Assessors should distinguish design effectiveness from operating effectiveness. A policy may be appropriately designed but not followed; a technical control may be deployed but exclude critical assets; a recovery plan may be complete but untested. The ontology currently records category implementation, so the assessment protocol must prevent a single document from being treated as sufficient evidence for a complex operational measure.

Conflicting evidence should be preserved rather than overwritten. For example, an approved policy may require MFA while an application inventory shows uncovered systems. The knowledge graph should represent both the policy claim and the observed exception, allowing the assessment conclusion to cite its basis. This approach supports auditability and avoids reducing compliance to self-attestation.

5.15 Evidence Quality Criteria

Criterion	Assessment Question	Example Strong Evidence	Example Weak Evidence
Relevance	Does the evidence directly support the measure and scoped system?	Configuration export tied to named assets	Generic vendor brochure
Authenticity	Can source and integrity be established?	Signed report from controlled repository	Unattributed screenshot
Currency	Is it recent enough for the control and risk?	Current-quarter access review	Undated historical document
Completeness	Does it cover the relevant population and exceptions?	Full asset report with exclusions explained	Small convenience sample
Consistency	Does it agree with other records and observations?	Policy, tickets, and configuration align	Policy contradicts deployed settings
Repeatability	Can another reviewer reproduce the result?	Documented query and retained dataset	Manual conclusion without method
Accountability	Is an owner responsible for the claim and remediation?	Named control owner and approval	Shared mailbox with no decision authority

5.16 Ethical and Governance Considerations

Compliance knowledge graphs may contain sensitive details about vulnerabilities, system architecture, suppliers, personnel, incidents, and control weaknesses. Data minimization and access control are therefore part of the research design, not merely deployment concerns. The graph should expose only the information required for the assessment purpose, and evidence repositories may need to remain separate from the semantic summary.

Automated classification can also create governance risk if decision makers treat a concise label as more authoritative than the underlying data. The interface should display missing evidence, assumptions, assessment dates, and confidence, and it should permit expert challenge. Responsibility for legal compliance remains with the regulated entity and its accountable management, not with the ontology or software.

Chapter 6: Ontology Design and Implementation

6.1 Ontology Structure and Namespace

The ontology is identified by the persistent IRI https://w3id.org/nis2/article21, ensuring long-term accessibility through the w3id.org persistent URI service. The ontology declaration in Turtle format specifies the version IRI https://w3id.org/nis2/article21/v1.0, Dublin Core metadata (title, creator, description, date), an owl:versionInfo string, owl:priorVersion, and an owl:imports declaration for the SKOS Core vocabulary. The ontology uses the namespace prefix nis2: for all local terms.

The ontology is maintained in Turtle for readable editing and RDF/XML for compatibility with OWL tools. The current server selects one serialization rather than comparing both graphs, so parity is a project maintenance requirement rather than an automated startup guarantee. Independent parsing should be used to confirm equivalence after ontology changes.

6.2 Core Classes

6.2.1 Entity Classes

The Entity class represents organizations subject to the modeled obligations. EssentialEntity and ImportantEntity are disjoint subclasses. Two entity examples are present: ExampleCompliantEntity, typed as EssentialEntity and linked to all twelve operational measure instances, and ExampleNonCompliantEntity, typed as ImportantEntity and linked to five.

CompliantEntity is equivalent to an intersection requiring Entity membership and at least one implementsMeasure value in each operational class. NonCompliantEntity is modeled as Entity intersected with the complement of CompliantEntity. Because OWL uses open-world semantics, the application’s procedural missing-category result should not be conflated with general OWL inference of negation.

6.2.2 Risk Management Measure Classes

RiskManagementMeasure is the superclass for TechnicalMeasure, OperationalMeasure, and OrganizationalMeasure, which are declared mutually disjoint. The twelve operational classes are subclasses of those three categories; they are not themselves declared mutually disjoint as one twelve-member set. Each has a label, comment, articleReference, and an example individual with quality properties and selected risk, incident, standard, or system relations.

6.2.3 Supporting Classes

The NetworkInformationSystem class represents the IT/OT systems subject to NIS2 scope. Three instances are provided: CoreBankingSystem, WebApplicationPlatform, and InternalITInfrastructure, representing typical essential entity system categories. The CybersecurityRisk class with subclasses DataBreachRisk, RansomwareRisk, InsiderThreatRisk, and SupplyChainCompromiseRisk represents risk categories. The SecurityStandard class with instances ISO27001, ISO27002, NISTFramework,

ENISAGuidelines, and CISControls represents the standards referenced in SKOS alignments and propertyChain inference.

6.3 Object Properties

Eight object properties are defined, each with rdfs:domain, rdfs:range, rdfs:label, and rdfs:comment annotations. The most significant are:

- implementsMeasure (Entity !' RiskManagementMeasure) — the primary compliance relationship used in the equivalentClass axiom.
- basedOnStandard (RiskManagementMeasure !' SecurityStandard) — connecting measures to their standard alignment, used in the property chain.
- usesStandard (Entity !' SecurityStandard) — derived property with owl:propertyChainAxiom [implementsMeasure, basedOnStandard].
- addressesRisk (RiskManagementMeasure !' CybersecurityRisk) — connecting measures to risk categories for CQ1 answering.
- preventsIncident (RiskManagementMeasure !' SecurityIncident) — connecting measures to incident individuals for CQ3 answering.
- appliesToSystem (RiskManagementMeasure !' NetworkInformationSystem) — connecting measure instances to the systems they protect.

The hasSubMeasure property is declared transitive. implementsMeasure and isImplementedBy are inverse properties. usesStandard is not asserted as transitive; it is derived through the two-step property chain from entity to measure to standard.

6.4 Data Properties

Five datatype properties are defined. riskLevel applies to CybersecurityRisk and stores one of low, medium, high, or critical. isAppropriate, isProportionate, and isStateOfTheArt apply to RiskManagementMeasure and are functional booleans. measureDescription provides a textual account of a measure. The SHACL graph checks these values more explicitly than OWL alone.

6.5 OWL 2 Axioms

The CompliantEntity equivalentClass axiom is the central formal contribution of the ontology. It defines:

$$CompliantEntity \equiv Entity \sqcap \exists implementsMeasure.RiskAnalysisPolicy \sqcap \exists implementsMeasure.IncidentHandling \sqcap \exists implementsMeasure.BusinessContinuityManagement \sqcap \ldots \sqcap \exists implementsMeasure.SecureCommunications$$

This intersection of twelve existential restrictions over implementsMeasure captures the legal requirement of Article 21(2) precisely: an entity is compliant if and only if it has at least one implemented instance of each of the twelve measure classes. The propertyChainAxiom on usesStandard:

$$usesStandard \text{ is inferred from the composition of } implementsMeasure \text{ and } basedOnStandard$$

enables the inference of security standard usage without requiring explicit usesStandard triples on entity instances. This demonstrates genuine OWL 2 reasoning capability beyond simple triple lookup.

RDF Serialization and OWL Compliance Axiom

nis2_article21_cybersecurity.ttl

Turtle serialization excerpt

```
@prefix : <https://w3id.org/nis2/article21#> .
@prefix owl: <http://www.w3.org/2002/07/owl#> .

:ExampleEncryption
    a owl:NamedIndividual, :Encryption ;
    rdfs:label "Example Encryption" ;
    :isAppropriate true ;
    :isProportionate true ;
    :basedOnStandard :NISTFramework ;
    :addressesRisk :DataBreachRisk ;
    :appliesToSystem :CoreBankingSystem .
```

RDF represents the measure as subject-predicate-object statements with typed resources and literals.

Figure 6.1: Turtle/RDF serialization of an example encryption measure

Source: excerpt adapted from nis2_article21_cybersecurity.ttl.

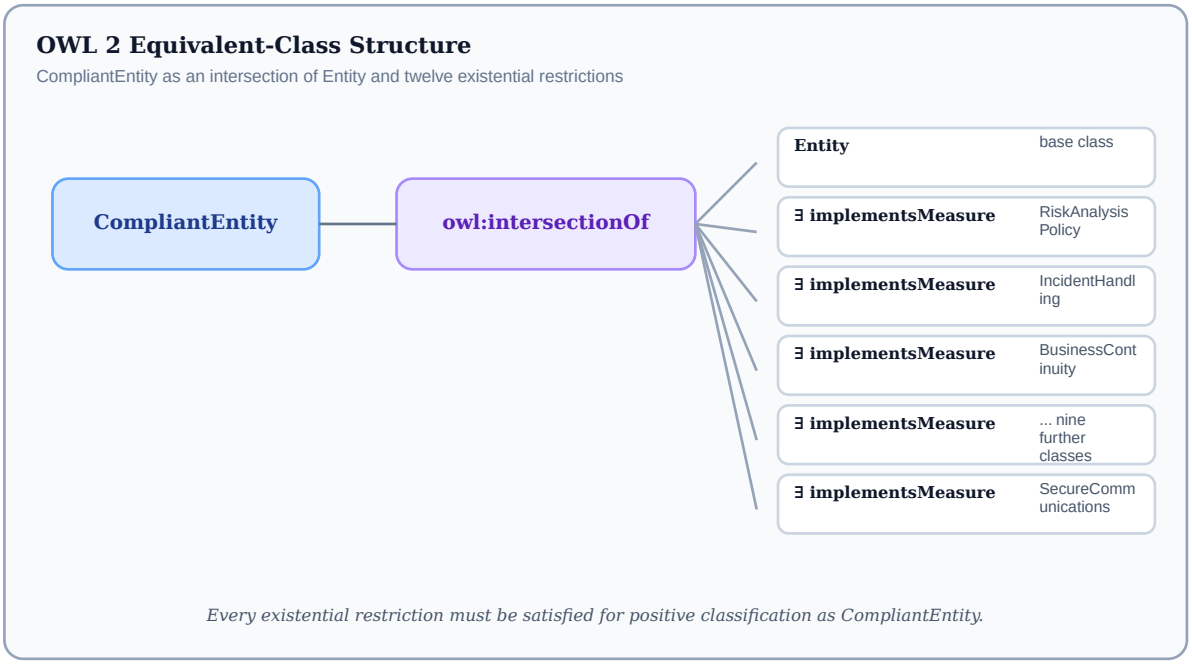

Figure 6.2: OWL equivalent-class structure for CompliantEntity

Source: visual interpretation of the owl:intersectionOf and owl:someValuesFrom axioms.

Additional OWL Illustrations

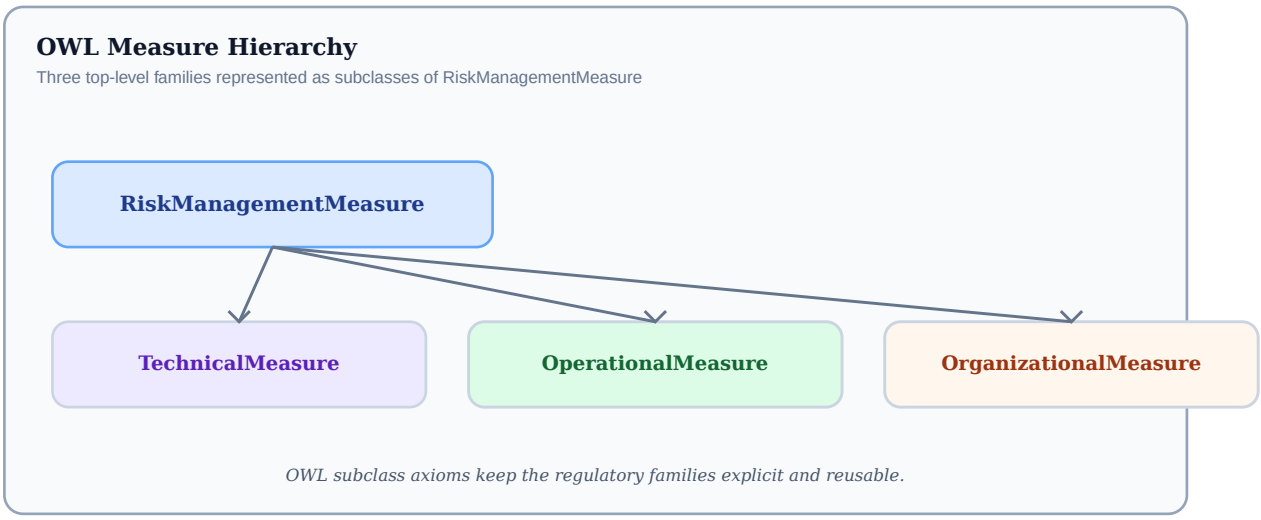

Figure 6.3: OWL subclass hierarchy for the three measure families

Source: schematic view of the measure taxonomy used in the ontology.

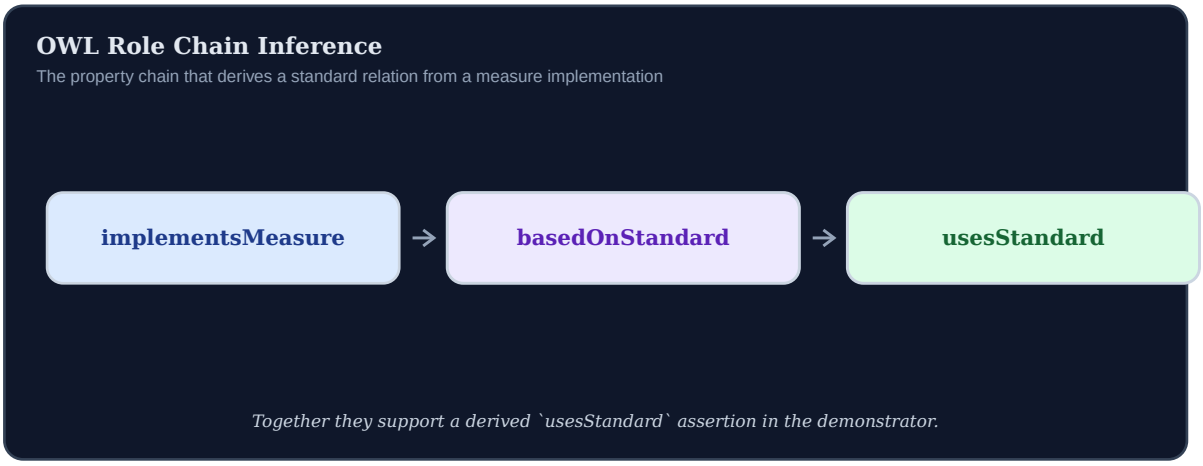

Figure 6.4: OWL role chain used to derive standard associations

Source: visual summary of the implementedMeasure and basedOnStandard chain.

6.6 SKOS Annotations and External Alignments

Eight SKOS alignment triples are added to measure classes, providing semantic interoperability with external cybersecurity knowledge bases:

Measure Class	SKOS Property	External Resource
Risk Analysis Policy	skos:close Match	ISO 27001 A.5 (Risk Assessment)
Incident Handling	skos:close Match	ENISA Incident Handling Guidelines
Supply Chain Security	skos:close Match	ISO 27002 A.5.19
Secure Development	skos:close Match	NIST CSF PR.IP-2
Basic Cyber Hygiene	skos:close Match	CIS Controls v8 IG1
Encryption	skos:exact Match	ISO 27001 A.8.24
Multi Factor Authentication	skos:close Match	NIST SP 800-63B
Human Resources Security	skos:close Match	ISO 27001 A.6

6.7 Competency Questions Validation

Prior to system integration, the five competency queries were checked against the

ontology graph. They exercise risk links, standard links, incident-prevention links, system scope, and entity-measure pairs. The two asserted entity examples provide the implementation pairs; additional boundary entities are generated through the real-time endpoint rather than stored in the ontology.

6.8 Ontology Validation

Validation combines parsing, structural checks, selected reasoning behavior, SHACL-oriented checks, and competency queries. ExampleCompliantEntity covers all twelve operational classes; ExampleNonCompliantEntity lacks seven. Direct Turtle parsing yields 526 triples. These results establish internal consistency for the tested patterns but do not replace OWL profile validation, complete reasoning, or execution by a general SHACL processor.

6.9 Conceptual Model Rationale

The model separates regulated entities, cybersecurity measures, information systems, risks, incidents, and standards. This prevents controls, obligations, and evidence from being treated as interchangeable concepts.

Explicit relations permit the same measure to participate in compliance classification, risk analysis, system scoping, and standards mapping without duplicating the underlying fact.

The model intentionally omits detailed evidence classes, which limits assurance-oriented use but keeps the core conceptualization manageable.

6.10 Compliance-Class Definition

CompliantEntity is defined as the intersection of Entity and twelve existential restrictions over implementsMeasure. The definition states that a compliant entity has at least one implemented measure in every operational class.

This declarative pattern places the criterion in the ontology rather than repeating twelve procedural checks in each consuming application. Organization-specific measure individuals can be introduced while the category-level condition remains stable.

Category membership alone cannot demonstrate control effectiveness or proportionality and must be interpreted with the validation limitations already identified.

6.11 Property-Chain Inference

The property chain states that an entity uses a standard when it implements a measure based on that standard. It eliminates repetitive entity-to-standard assertions and produces a visible inference path through the relevant measure.

This pattern demonstrates semantic enrichment beyond triple retrieval: independently meaningful assertions combine to derive additional knowledge suitable for reporting and cross-framework analysis.

The inference indicates a relationship to a standard, not certification against that standard or full implementation of all its controls.

6.12 Disjointness and Consistency

Disjointness axioms document intended conceptual boundaries between entity categories and among the twelve legal measure classes. They help expose unintended multiple typing and support disciplined ontology maintenance.

The axioms operate at the chosen abstraction level. If a future evidence model represents one technical control as satisfying several legal categories, the control artifact should be distinguished from the category-specific measure realizations.

Overly strong disjointness can create false inconsistencies, so these axioms require review as the ontology becomes more detailed.

6.13 Namespace and Serialization Strategy

Turtle is the primary authoring format because it is concise and reviewable, while RDF/XML supports compatibility with established OWL tools. Both serializations identify the same ontology and are intended to encode equivalent graphs.

The persistent w3id.org namespace separates resource identity from the current repository location. Stable identifiers support citation, external linking, versioning, and long-term reuse.

Maintaining two serializations creates a parity obligation and should eventually be automated through generation from one canonical source.

6.14 Formal Semantics of the Compliance Pattern

The CompliantEntity definition contains Entity plus twelve existential restrictions in one owl:intersectionOf list. Each restriction requires at least one implementsMeasure value belonging to the corresponding measure class. Under OWL semantics, an individual satisfying every conjunct can be classified as CompliantEntity even when that type is not asserted explicitly. The definition is reusable because the measure values may be any individuals of the required classes; they need not be the twelve example individuals included in the ontology.

The universal restriction attached to Entity states that every implementsMeasure value of an Entity is a RiskManagementMeasure. This does not create a measure, impose a minimum number of values, or validate a closed list. It constrains the type of values when the relation is known. The existential restrictions in CompliantEntity provide the positive minimum conditions. SHACL provides the closed-world reporting behavior when submitted entity data omits one or more categories.

The NonCompliantEntity definition requires special care. It is the intersection of Entity and the complement of CompliantEntity. Under the open-world assumption, failure to prove that an individual is compliant is not generally sufficient to infer that it belongs to the complement. A complete reasoner would need enough information to establish non-membership. The JavaScript service instead treats the locally observed measure set as complete and reports missing categories procedurally. The report therefore distinguishes OWL entailment from application-level closed-world classification.

6.15 Axiom-to-Requirement Traceability Matrix

Ontology Construct	Implemented Form	Research Purpose	Semantic Boundary
Entity range restriction	implements Measure only Risk Management Measure	Constrains the type of known implemented measures	Does not require any measure to exist
Compliant Entity equivalence	Entity intersected with 12 some Values From restrictions	Defines sufficient and necessary category coverage	Does not evaluate quality or evidence
Non Compliant Entity equivalence	Entity intersected with complement Of Compliant Entity	Represents logical non-compliance	Non-membership is difficult to infer under open-world semantics
Entity-category disjointness	Essential Entity disjoint With Important Entity	Prevents incompatible regulatory typing	Assumes categories are mutually exclusive in the modeled context
Measure-category disjointness	Technical, Operational, Organizational in All Disjoint Classes	Makes high-level measure categories exclusive	May be too strong for multi-nature controls if controls and legal realizations are conflated
Inverse property	implements Measure inverse Of is Implemented By	Supports navigation in both directions	Inverse facts require a reasoner or query rewriting if not materialized
Property chain	implements Measure o based On Standard -> uses Standard	Derives entity-standard relations	Indicates relation, not certification
Transitive property	has Sub Measure transitive	Supports nested decomposition	No sub-measure instances are currently asserted
Functional quality properties	At most one boolean value per measure	Avoids contradictory duplicate values	A missing value remains possible in OWL
SKOS mappings	exact Match or close Match links	Supports cross-framework navigation	Mapping strength needs expert governance

6.16 SHACL Constraint Semantics

Article21ComplianceShape targets Entity and contains twelve qualified minimum-count constraints over implementsMeasure. Each constraint tests whether at least one value conforms to a class-specific value shape. This structure is more precise than a simple minimum count of twelve because twelve values of the same class would not satisfy the remaining eleven categories. The shape also requires all implemented values to be RiskManagementMeasure instances, providing a structural check aligned with the ontology range.

MeasureQualityShape targets RiskManagementMeasure and combines datatype, minimum-count, maximum-count, and hasValue constraints for isAppropriate, isProportionate, and isStateOfTheArt. The severity is Warning rather than Violation, reflecting that these assertions concern qualitative assessment. The shape also requests a textual measureDescription at Info severity. This severity design separates mandatory category coverage from documentation and quality warnings, although an organization may adopt stricter local policy.

RiskLevelShape targets CybersecurityRisk and requires exactly one string selected from low, medium, high, or critical. The controlled list improves consistency for queries and visual displays. It remains a simplified ordinal representation: no scoring methodology, likelihood, impact, time horizon, or assessment date is modeled. A production risk

ontology would need these dimensions and explicit provenance.

6.17 Ontology Inventory and Internal Consistency

Direct parsing of the Turtle source yields 526 triples. The graph declares 28 OWL classes, 9 object properties, 5 datatype properties, 1 annotation property, 31 named individuals, 13 restrictions, 4 functional-property declarations, and 1 transitive-property declaration. The named individuals include five standards, five risks, four incidents, twelve example measures, two entity examples, and three network and information systems.

These counts are useful regression indicators but not quality measures by themselves. A larger graph is not necessarily a better ontology. Their value lies in detecting unexpected structural changes: a missing measure class, a lost restriction, or a changed number of example instances can trigger review. Counts should be combined with semantic tests, shape validation, competency questions, and independent reasoning.

6.18 Risk and System Coverage in the Example Graph

Resource	Type / Level	Connected Measures	Analytical Observation
Data Breach Risk	Cybersecurity Risk / high	Risk Analysis Policy, Effectiveness Assessment, Encryption, Multi Factor Authentication, Secure Communications	Broad coverage across governance, assurance, cryptography, authentication, and communications
Ransomware Risk	Cybersecurity Risk / critical	Incident Handling	Current graph emphasizes response; prevention and recovery links could be expanded
DDo SRisk	Cybersecurity Risk / high	Business Continuity Managemen	Focuses on impact and recovery rather than preventive network controls
Insider Threat Risk	Cybersecurity Risk / medium	Risk Analysis Policy, Basic Cybe Hygiene, Training Awareness, Human Resources Security	Combines governance, operational baseline, awareness, and personnel controls
Supply Chain Compromise Risk	Cybersecurity Risk / critical	Supply Chain Security, Secure Development	Connects supplier governance with technical lifecycle practices
Core Banking System	Network Information System	Risk Analysis Policy, Incident Handling, Business Continuity Management, Encryption, Multi Factor Authentication	Receives the broadest explicit system coverage
Web Application Platform	Network Information System	Incident Handling, Secure Development, Encryption	Coverage centers on software lifecycle, protection, and response
Internal ITInfrastructure	Network Information System	Risk Analysis Policy, Incident Handling, Multi Factor Authentication	Coverage could be extended to hygiene, continuity, and encryption

The mapping demonstrates the graph’s analytical potential but also reveals sparsity. A measure without appliesToSystem does not necessarily lack scope; the relation may simply be unasserted. Under open-world semantics the graph cannot assume that the listed systems are exhaustive. A production assessment should model system inventories, ownership, criticality, dependencies, and the explicit scope of each measure.

Chapter 7: System Implementation

7.1 System Architecture Overview

The system follows a three-tier REST architecture. At startup, the Node.js/Express server selects the RDF/XML ontology when that file exists and otherwise falls back to Turtle. Parsed quads are cached together with an N3.Store and reloaded when the selected file modification time changes. The server exposes six functional endpoints, including graph-data delivery, and serves the static frontend from the public directory.

The architecture deliberately avoids external dependencies such as Apache Jena, GraphDB, or Stardog, enabling deployment on any Node.js-capable environment without database installation or configuration. The entire system runs from a single directory with npm install and node server.js.

7.2 Backend Implementation

7.2.1 RDF/OWL Parsing (N3.js)

Turtle is parsed with N3.Parser, while RDF/XML is parsed through rdf-parse. The selected file is converted into an in-memory N3.Store for basic graph-pattern execution. The store-construction code skips unsupported term structures in a try/catch block, so the original parsed quad array remains the more complete representation for structural inspection.

7.2.2 API Endpoints

Six application endpoints are implemented:

- GET /api/ontology — transforms ontology resources and selected relations into graph nodes and edges for visualization.
- GET /api/validate — analyzes the loaded ontology and returns class count, property count, instance count, axiom types present, and a structural validity assessment.
- GET /api/reason — applies bounded category-coverage logic to asserted entity-measure links and derives usesStandard relations.
- GET /api/shacl — performs JavaScript checks corresponding to the three local shape designs.
- GET /api/sparql — accepts a SELECT query through the query parameter, parses it with sparqljs, and evaluates supported basic graph patterns and filters.
- POST /api/check-entity — accepts {entityName, entityType, implementedMeasures[]} and returns a complete compliance assessment including score, missing measures, inferred standards, risks addressed, and OWL-inferred class.

7.2.3 Reasoning Engine

The reasoning engine is a structural implementation of the selected compliance pattern. It maps each example measure individual to one required class, gathers asserted implementsMeasure links, compares the resulting set with REQUIRED_MEASURES, and reports complete or incomplete coverage. It also composes implementsMeasure with basedOnStandard and records the intermediate measure as provenance. The service does not execute arbitrary OWL axioms.

7.2.4 SHACL Validation Logic

The SHACL-oriented endpoint implements three fixed checks in JavaScript. It evaluates operational-class coverage for Entity instances, true boolean quality values and descriptions for measure individuals, and controlled riskLevel values for CybersecurityRisk individuals. It returns shape names, focus nodes, severities, and messages, but it does not parse arbitrary SHACL graphs or implement the complete SHACL specification.

7.2.5 Property Chain Inference (usesStandard)

The property chain usesStandard is implemented by composing implementsMeasure with basedOnStandard. For a given entity E with implementsMeasure triples to measure instances M •...M ™, the engine queries the store for basedOnStandard triples on each M b. For each found triple M b basedOnStandard S, it adds an inferred usesStandard triple (E, usesStandard, S) to the reasoning result. The provenance is recorded as {standard: S, via: M b} in the inferredStandards array returned to the client. This implementation directly reflects the OWL 2 propertyChainAxiom semantics without requiring a full OWL reasoner.

7.3 Frontend Implementation

7.3.1 Interactive Knowledge Graph Visualization

The knowledge graph visualization uses the vis-network library (v9) to render the OWL ontology as an interactive force-directed graph. Nodes represent OWL classes, instances, and literals, colored by type: classes in blue (#3498db), instances in green (#27ae60), literals in orange (#f39c12), and measure classes in purple (#8e44ad). Edges represent properties, labeled with the property local name. Users can drag nodes, zoom, and click to inspect node metadata. The graph is rendered from the /api/validate response, which includes all classes and instances with their properties.

7.3.2 User Interface Design

The UI is a single-page application with a fixed toolbar containing seven action buttons: Load Ontology, Run Reasoner, SHACL Validation, SPARQL Query, Class Hierarchy, Check Real Entity, and Export. Each button opens a modal panel overlaying the graph visualization. The color scheme uses semantic colors: green for compliant/success states, red for non-compliant/error states, purple for SHACL, orange for the entity checker, and blue for general actions. The interface uses CSS Grid for responsive layout and custom CSS for the compliance result cards, validation badges, and standard tags.

7.3.3 Real-Time Validation

The real-time entity compliance checking panel allows users to enter any entity name, select entity type (Essential/Important), and check or uncheck any of the twelve Article 21(2) measures. On submission, the frontend calls POST /api/check-entity and renders the result including: a green/red compliance badge, three metric cards (compliance score %, implemented count, OWL inferred class), a missing measures list with legal references, inferred security standards as colored tags, risk categories addressed, and the legal basis citation.

7.4 Integration and Testing

Integration testing verified end-to-end behavior of all five API endpoints using curl and

automated Node.js test scripts. Key test scenarios included: (1) full compliance check for an entity implementing all 12 measures — expected: 100%, CompliantEntity; (2) partial compliance check with 3 measures — expected: 25%, NonCompliantEntity, 9 missing measures; (3) shape-specific structural validation against ExampleNonCompliantEntity — expected: Article21ComplianceShape FAIL with 7 violation messages; (4) limited SPARQL SELECT query CQ2 for ExampleCompliantEntity — expected: 5 standards returned; (5) empty entity name submission — expected: validation error response. All test scenarios passed successfully. These tests confirm the prototype behavior; they do not certify standards-complete OWL reasoning, general SHACL conformance, or full SPARQL 1.1 compliance.

7.5 Architectural Separation of Concerns

The implementation separates ontology data, validation shapes, backend services, and presentation logic. This permits the formal artifact to be inspected independently and prevents interface behavior from becoming the sole source of compliance logic.

The backend adapts RDF representations into structured JSON while retaining legal references, class names, missing measures, and inference paths. Each endpoint corresponds to a distinct analytical responsibility.

Some reasoning remains procedural, so full semantic independence from application code has not yet been achieved.

7.6 Reasoning-Service Implementation

The reasoning service retrieves implemented measures, compares their types with the twelve required categories, and reports classification, score, gaps, standards, and risks.

The implementation is deterministic and transparent for the patterns used in this study. It operationalizes the ontology without requiring an external semantic platform.

The service is not a general OWL 2 DL reasoner, and unsupported constructs must not be assumed to have been evaluated.

7.7 Validation and Error Semantics

Input validation protects interpretation of results. Entity requests are normalized against recognized identifiers, malformed queries return explicit errors, and ontology-loading failures prevent dependent operations from reporting misleading success.

Observable failure behavior matters because an empty or partial response could otherwise be mistaken for the absence of an obligation or risk.

Production use would additionally require authentication, authorization, audit logging, rate limits, and stronger request schemas.

7.8 User-Interface Design Rationale

The interface provides graph exploration, metrics, query tables, validation reports, and a task-oriented entity checker. These views support users with different levels of Semantic Web expertise.

Readable labels appear alongside formal identifiers and legal references. Missing measures are listed explicitly rather than hidden behind a percentage score, preserving explainability.

Functional demonstration does not replace a controlled usability study with compliance practitioners.

7.9 Endpoint-Level Implementation Analysis

The service boundary is small enough to examine endpoint by endpoint. This analysis is based on the implemented routes rather than an idealized architecture. It clarifies what each endpoint computes, which ontology features it uses, and which semantic claims it cannot support.

7.9.1 GET /api/ontology

Transforms parsed RDF into nodes and edges for the browser. It preserves labels, comments, selected properties, subclass links, and local object-property relations.

Interpretive boundary: Useful for exploration, but blank-node OWL structures are not rendered as full logical expressions and should be inspected in the ontology source or a dedicated OWL editor.

7.9.2 GET /api/validate

Checks parse success, ontology declaration, equivalent-class and disjointness axioms, presence of the twelve measure classes, and cycles in named subclass edges. It reports counts for classes, properties, instances, and hierarchy edges.

Interpretive boundary: This is a structural validator. It does not perform model-theoretic consistency checking, datatype reasoning, profile validation, or comparison of Turtle and RDF/XML graphs.

7.9.3 GET /api/sparql

Parses SELECT queries with sparqljs and evaluates basic graph patterns plus a limited set of filters over an N3 store. Results are returned using a SPARQL-JSON-like structure.

Interpretive boundary: OPTIONAL, UNION, aggregation, property paths, subqueries, named graphs, ordering, and many expression forms are not executed, even if some can be parsed.

7.9.4 GET /api/reason

Maps measure individuals to one of the twelve required classes, gathers implementsMeasure assertions, classifies complete and incomplete entities, checks entity-type overlap, and derives usesStandard links.

Interpretive boundary: The procedure mirrors selected ontology patterns but is not a

complete OWL reasoner. In particular, absence of a measure is handled procedurally rather than inferred through open-world complement semantics.

7.9.5 GET /api/shacl

Implements the three local shape intentions in JavaScript: category coverage for entities, quality booleans and descriptions for measures, and controlled risk levels.

Interpretive boundary: It is not a general SHACL engine and does not parse and execute arbitrary shapes from the shapes graph. Results should be described as shape-specific structural validation.

7.9.6 POST /api/check-entity

Validates a user-supplied entity name, type, and measure list; removes unknown measure identifiers; calculates coverage; and derives standards and addressed risks from the example measure relations.

Interpretive boundary: It assesses submitted category claims only. It does not persist the entity, validate evidence, or check whether the selected controls are appropriate, proportionate, effective, and state of the art.

Endpoint	Method	Implemented Responsibility	Principal Boundary
/ api/ ontology	GET	Transforms parsed RDF into nodes and edges for the browser. It preserves labels, comments, selected properties, subclass links, and local object-property relations.	Useful for exploration, but blank-node OWL structures are not rendered as full logical expressions and should be inspected in the ontology source or a dedicated OWL editor.
/ api/ validate	GET	Checks parse success, ontology declaration, equivalent-class and disjointness axioms, presence of the twelve measure classes, and cycles in named subclass edges. It reports counts for classes, properties, instances, and hierarchy edges.	This is a structural validator. It does not perform model-theoretic consistency checking, datatype reasoning, profile validation, or comparison of Turtle and RDF/ XML graphs.
/ api/ sparql	GET	Parses SELECT queries with sparqljs and evaluates basic graph patterns plus a limited set of filters over an N3 store. Results are returned using a SPARQL-JSON-like structure.	OPTIONAL, UNION, aggregation, property paths, subqueries, named graphs, ordering, and many expression forms are not executed, even if some can be parsed.
/ api/ reason	GET	Maps measure individuals to one of the twelve required classes, gathers implements Measure assertions, classifies complete and incomplete entities, checks entity-type overlap, and derives uses Standard links.	The procedure mirrors selected ontology patterns but is not a complete OWL reasoner. In particular, absence of a measure is handled procedurally rather than inferred through open-world complement semantics.
/ api/ shacl	GET	Implements the three local shape intentions in JavaScript: category coverage for entities, quality booleans and descriptions for measures, and controlled risk levels.	It is not a general SHACL engine and does not parse and execute arbitrary shapes from the shapes graph. Results should be described as shape-specific structural validation.

Endpoint	Method	Implemented Responsibility	Principal Boundary
/ api/ check-entity	POST	Validates a user-supplied entity name, type, and measure list; removes unknown measure identifiers; calculates coverage; and derives standards and addressed risks from the example measure relations.	It assesses submitted category claims only. It does not persist the entity, validate evidence, or check whether the selected controls are appropriate, proportionate, effective, and state of the art.

Chapter 8: System Demonstration and Use Cases

8.1 Interactive Graph Visualization

Upon loading the application at http://localhost:3000, the user is presented with an interactive graph derived from /api/ontology. It includes Entity and its regulatory subclasses, the RiskManagementMeasure hierarchy and twelve operational classes, supporting risk, incident, system, and standard resources, and selected local object-property edges. The visualization is an explanatory projection of the RDF graph rather than a complete rendering of blank-node OWL expressions.

The visualization provides direct insight into the ontology structure that would otherwise require reading through hundreds of lines of Turtle syntax. For a professor presentation, the graph effectively communicates the complete NIS2 Article 21 compliance domain model in a single view.

Ontology Explorer and Representative Relations

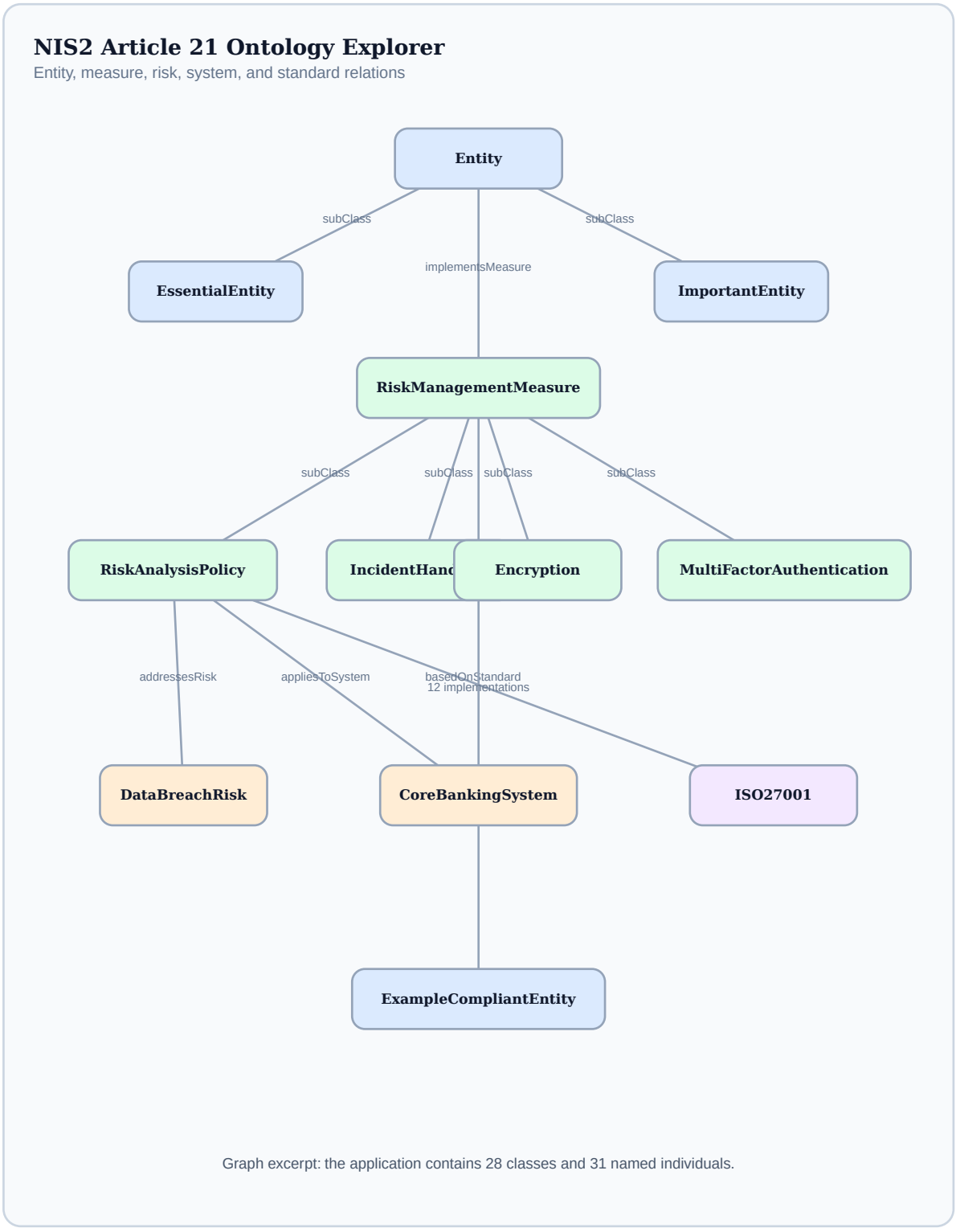

Figure 8.1: Ontology Explorer and Representative Relations

Source: generated from the ontology vocabulary and relations used by GET /api/ontology.

8.2 OWL Validation and Reasoning

Clicking "Load Ontology" triggers /api/validate, which reports parse status, selected axiom checks, the presence of all twelve operational classes, hierarchy-cycle results, and structural counts. Independent Turtle parsing gives 526 triples, 28 OWL classes, 9 object properties, 5 datatype properties, and 31 named individuals. The endpoint is a structural validator and should not be described as proof of full OWL 2 DL conformance.

Clicking "Run Reasoner" triggers /api/reason. ExampleCompliantEntity covers all twelve operational classes, while ExampleNonCompliantEntity covers five and is reported with seven gaps. The panel also shows deduplicated standards derived through the implementsMeasure/basedOnStandard path. Only these two entity examples are asserted in the ontology.

Structural Ontology Validation Report

Figure 8.2: Structural Ontology Validation Report

Source: verified output from GET /api/validate on the 526-triple RDF/XML serialization.

Entity Classification and Derived Standards

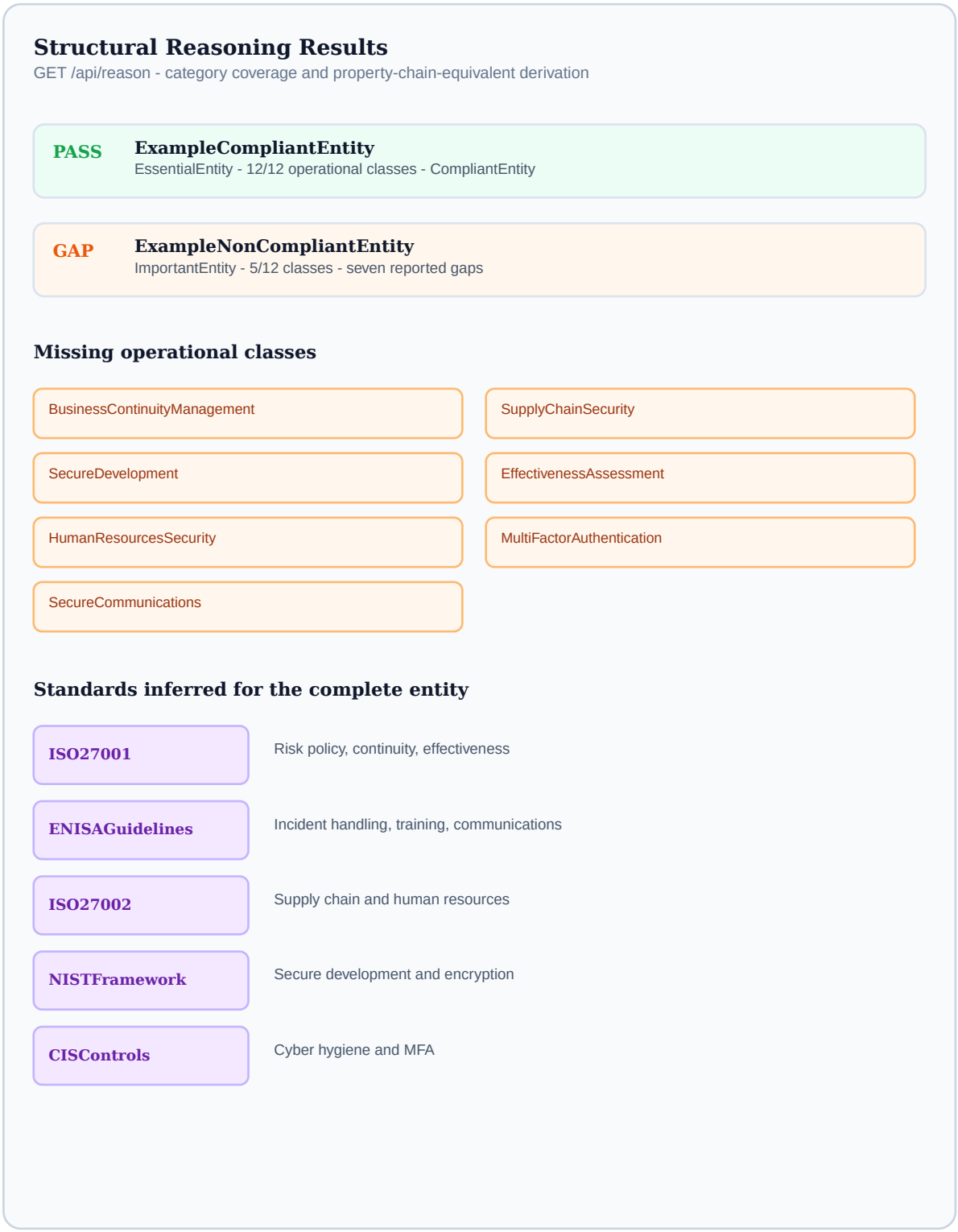

Figure 8.3: Entity Classification and Derived Standards

Source: verified output from GET /api/reason; the panel represents bounded structural reasoning.

8.3 SHACL Shapes Validation

Clicking "SHACL Validation" triggers /api/shacl. The coverage check passes for ExampleCompliantEntity and reports seven missing operational classes for ExampleNonCompliantEntity. All twelve example measure individuals contain true values for the three quality properties and a measureDescription. All five risk individuals contain one permitted riskLevel value.

SHACL-Oriented Validation Results

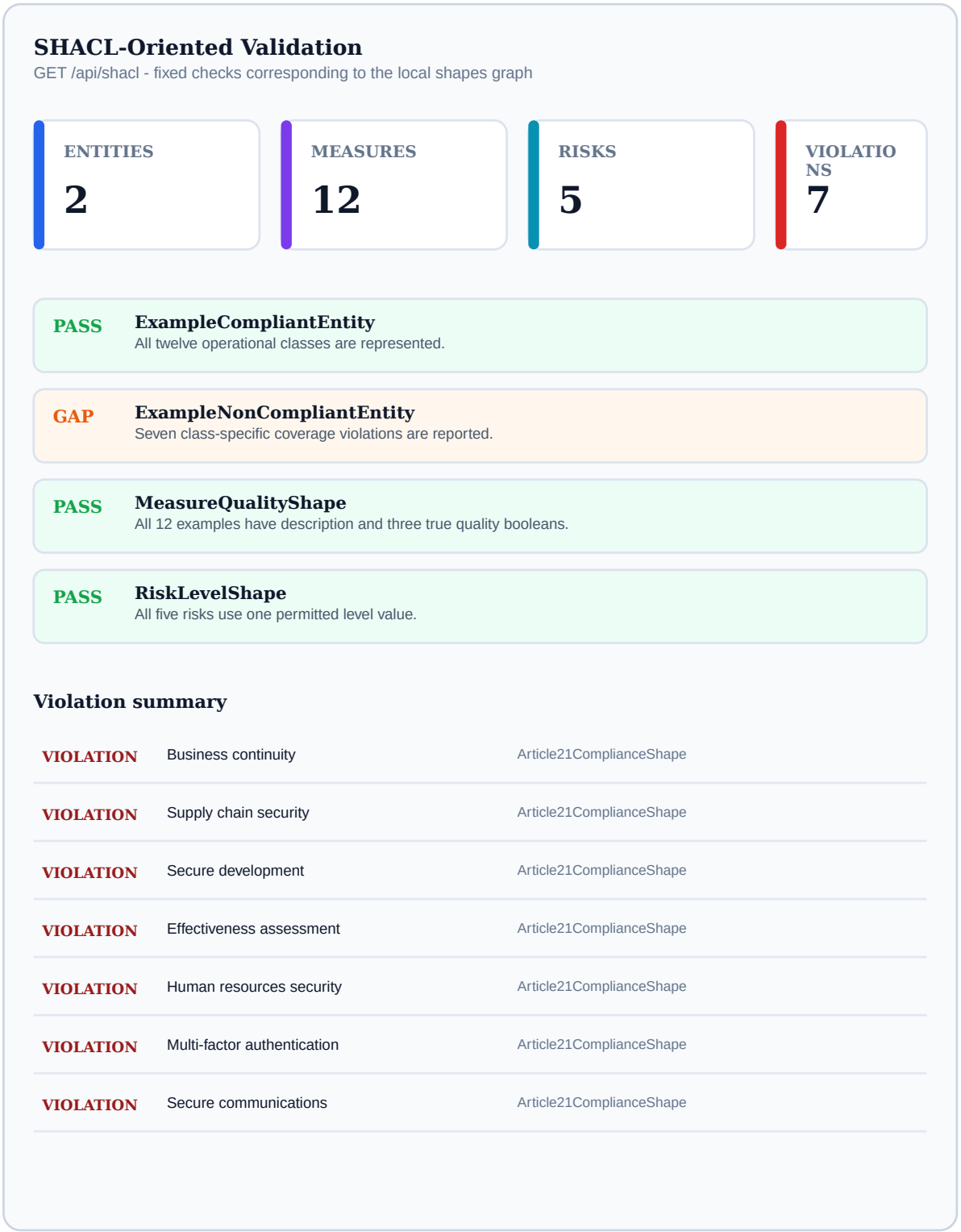

Figure 8.4: SHACL-Oriented Validation Results

Source: verified output from GET /api/shacl; full SHACL conformance requires an independent processor.

8.4 SPARQL Query Interface

The SPARQL panel provides five pre-loaded competency question queries accessible via CQ1–CQ5 buttons, plus a free-text editor for custom queries. CQ1 returns measures addressing critical or high risks: RiskAnalysisPolicy, IncidentHandling, SupplyChainSecurity, and Encryption. CQ2 returns the five standards used by ExampleCompliantEntity through property chain inference. CQ3 returns measures that

preventsIncident for DataBreachIncident and RansomwareIncident. CQ4 returns measures that appliesToSystem CoreBankingSystem. CQ5 returns all entity-measure implementation pairs as a table. Results are rendered in a formatted table with alternating row colors and column headers.

SPARQL Competency Query

CQ2 - standards associated with the complete example entity

SPARQL editor

```
PREFIX nis2: <https://w3id.org/nis2/article21#>
SELECT ?entity ?standard
WHERE {
  ?entity a nis2:CompliantEntity .
  ?entity nis2:implementsMeasure ?measure .
  ?measure nis2:basedOnStandard ?standard .
}
```

Query results

entity	standard
ExampleCompliantEntity	ISO27001
ExampleCompliantEntity	ENISAGuidelines
ExampleCompliantEntity	ISO27002
ExampleCompliantEntity	NISTFramework
ExampleCompliantEntity	CISControls

Five unique standards are reached through implemented measures and their basedOnStandard relations.

Figure 8.6: SPARQL Competency Query and Results

Source: CQ2 graph pattern and the five standard resources associated with ExampleCompliantEntity.

8.5 Real-Time Entity Compliance Checking

The "Check Real Entity" panel demonstrates the ontology's applicability beyond the pre-loaded example instances. A user enters an entity name (e.g., "MyBank"), selects type "Essential Entity", and checks the twelve measure checkboxes corresponding to their implemented controls. The panel pre-checks all twelve measures to illustrate the fully-compliant case. On submission, the compliance result renders:

- Green badge: "' COMPLIANT — MyBank (EssentialEntity)"
- Score cards: 100% compliance score, 12/12 measures, "CompliantEntity" OWL inferred class
- Inferred standards: ISO27001, ISO27002, NISTFramework, ENISAGuidelines,

CISControls (purple tags)

- Risks addressed: DataBreachRisk, InsiderThreatRisk, RansomwareRisk, SupplyChainCompromiseRisk (blue tags)
- Legal basis: "Directive (EU) 2022/2555, Article 21(2)"

When six measures are unchecked (simulating a partial compliance scenario), the result shows a red "' NON-COMPLIANT" badge, a 50% score, and lists the six missing measures with their Article 21(2) legal references. This use case demonstrates that the ontology functions as a compliance checking engine for any organization subject to NIS2, not merely for the pre-defined example entities.

Real-Time Entity Compliance Assessment

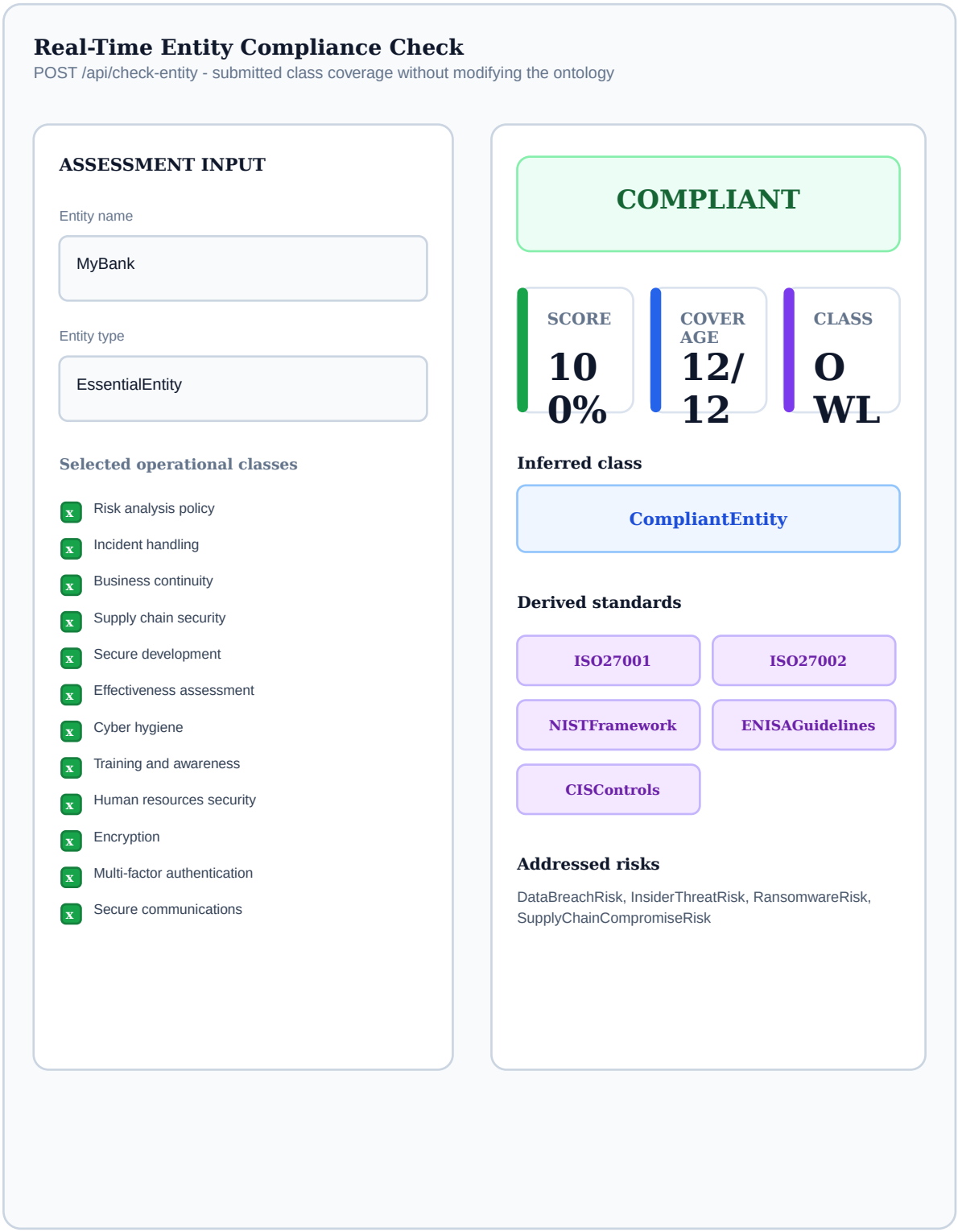

Figure 8.5: Real-Time Entity Compliance Assessment

Source: generated from the request and response structure of POST /api/check-entity for a complete EssentialEntity example.

8.6 End-to-End Assessment Scenario

An assessment begins with an entity type and asserted measure implementations. The backend normalizes the submission, evaluates category coverage, derives related standards and risks, and returns a structured result.

The workflow demonstrates how the components interact: the ontology supplies vocabulary and relations, SHACL supplies constraints, the service executes bounded reasoning, and the interface makes the outcome reviewable.

The scenario uses self-declared input and does not establish authenticity or sufficiency of implementation evidence.

8.7 Interpretation of Demonstration Results

A complete result means that the submitted graph contains at least one implementation for every modeled category and satisfies the prototype's structural checks. It does not establish effectiveness, proportionality, or regulatory approval.

A missing category identifies a gap in represented information. Remediation may involve implementing a control, documenting an existing control, correcting a mapping, or gathering evidence.

Human review remains necessary, so the demonstrator should be read as decision support rather than automated legal certification.

8.8 Scenario A: Core Banking Service Assessment

A banking assessment can begin with CoreBankingSystem as the critical service-supporting system. The example graph connects that system to RiskAnalysisPolicy, IncidentHandling, BusinessContinuityManagement, Encryption, and MultiFactorAuthentication. These relations cover governance, response, resilience, data protection, and access control. They provide a useful starting view of which modeled measures explicitly name the system.

The view is not complete enough for an audit. The assessor would also expect supply-chain controls for hosted components, secure-development controls for internally developed software, effectiveness testing, hygiene, personnel security, and secure communications to have a defined scope. Their absence from appliesToSystem does not prove that they are absent operationally. It identifies where explicit scope data should be added. This demonstrates how a knowledge graph can reveal documentation gaps without prematurely treating them as control failures.

Evidence could include system ownership, architecture, criticality, data classification, recovery objectives, identity-provider configuration, encryption status, incident playbooks, dependencies, and recent test results. Each evidence item should support a specific measure-system assertion and carry a date and owner. The resulting graph would permit queries such as which critical systems lack tested recovery evidence or which privileged-access controls have expired assessments.

8.9 Scenario B: Ransomware Preparedness and Response

RansomwareRisk is marked critical and linked directly to ExampleIncidentHandling. The incident-handling individual prevents RansomwareIncident and minimizes ServiceDisruptionIncident. This captures response and impact reduction but leaves several important preventive and recovery relations implicit. BasicCyberHygiene, MultiFactorAuthentication, SecureDevelopment, BusinessContinuityManagement, and TrainingAwareness can all contribute to a ransomware scenario even though the current graph does not connect each one to RansomwareRisk.

An expanded scenario would model the attack chain: initial access, privilege escalation, lateral movement, encryption, service disruption, and recovery. Measures could then be linked to the phases they prevent, detect, or mitigate. Evidence would include phishing resistance, endpoint detection coverage, privileged-access controls, network segmentation, immutable backup tests, incident exercises, and restoration timing. This richer representation would support defense-in-depth analysis instead of assigning one risk to one primary measure.

The scenario also illustrates the difference between control presence and operational resilience. A backup policy may exist while restoration fails; MFA may be enabled for remote access but absent on legacy administration; an incident plan may exist without current contacts. The ontology should eventually represent assessment findings and exceptions so that contradictory evidence can coexist with policy claims.

8.10 Scenario C: Software and Supplier Compromise

SupplyChainCompromiseRisk is linked to SupplyChainSecurity and SecureDevelopment, connecting organizational supplier governance with technical lifecycle security. This pairing is analytically important because modern software risk crosses organizational boundaries. A supplier can provide a compromised update, while weak internal build and deployment controls can allow that compromise to propagate.

A realistic assessment would identify critical suppliers, software services, libraries, build systems, repositories, signing infrastructure, deployment pipelines, and externally hosted components. SupplyChainSecurity evidence would cover due diligence, contracts, assurance, access, notification, and concentration risk. SecureDevelopment evidence would cover dependency scanning, software bills of materials, code review, build isolation, artifact signing, testing, patching, and vulnerability disclosure.

The current ontology can express that both measure categories address the same risk, but it cannot represent a dependency chain or identify which supplier affects which system. Adding Supplier, Product, Component, Contract, Dependency, and AssuranceReport classes would enable targeted queries. For example, the organization could ask which critical systems depend on suppliers without current assurance or which components with known vulnerabilities lack an approved remediation decision.

8.11 Scenario D: Joiner-Mover-Leaver Control

HumanResourcesSecurity and TrainingAwareness address different but related aspects of personnel risk. A joiner requires screening where lawful, confidentiality obligations, role assignment, least-privilege access, asset issuance, and initial training. A mover requires entitlement reassessment and role-specific training. A leaver requires timely access revocation, asset return, transfer of responsibilities, and retention of necessary records.

The example graph links HumanResourcesSecurity to InsiderThreatRisk and

DataBreachIncident and links TrainingAwareness to InsiderThreatRisk. A more detailed graph would represent employment events, roles, accounts, assets, approvals, and completion records. SHACL could then validate that every termination event has revocation and asset-return evidence, while SPARQL could identify overdue access reviews or personnel in privileged roles without current training.

This scenario demonstrates why one boolean implementation claim is insufficient. Personnel controls are processes whose effectiveness depends on timeliness and population coverage. Evidence must be sampled or monitored across events. The ontology can provide the shared vocabulary, but operational systems remain the source of identity, HR, ticket, and training records.

8.12 Scenario E: Audit Preparation and Management Review

Before an audit or management review, the graph can be used to organize the evidence request. Each operational class provides a top-level assessment domain. Relations to systems, risks, incidents, and standards help explain scope and cross-framework relevance. SHACL can identify missing structural claims, while queries can list measures, evidence links, owners, and systems once those concepts are added.

The process should not begin by selecting all twelve checkboxes. It should begin with evidence collection and evaluation. Assessors determine whether the evidence supports implementation within scope, record exceptions and findings, and only then assert or approve the corresponding measure realization. This ordering reduces confirmation bias and turns the graph into an assessment record rather than a self-attestation form.

Management review needs an aggregate view without losing traceability. A dashboard may show category coverage, overdue findings, evidence age, critical system gaps, and risk concentration. Every aggregate should link back to underlying observations. The present prototype demonstrates the category layer; evidence and governance extensions are the next step required for credible audit use.

8.13 Scenario Comparison

Scenario	Primary Classes	Additional Data Needed	Decision Supported
Core banking	Risk Analysis Policy, Incident Handling, BCM, Encryption, MFA	System criticality, dependencies, recovery and configuration evidence	System-level coverage review
Ransomware	Incident Handling plus preventive and recovery measures	Attack phases, control coverage, exercise and restoration results	Defense-in-depth and recovery readiness
Supply chain	Supply Chain Security, Secure Development	Suppliers, products, components, contracts, assurance	Third-party and software dependency risk
Personnel lifecycle	Human Resources Security, Training Awareness	People, roles, accounts, events, assets, training records	Access and personnel process assurance
Audit preparation	All operational classes	Evidence, owners, findings, dates, approvals	Traceable management and audit reporting

Chapter 9: Evaluation and Results

9.1 Ontology Completeness

Ontology completeness was assessed against the ten legal categories and their decomposition into twelve operational classes. Every operational class has a label, comment, articleReference, and example individual. Eight classes currently carry direct SKOS mappings: RiskAnalysisPolicy, IncidentHandling, BusinessContinuityManagement, SupplyChainSecurity, SecureDevelopment, BasicCyberHygiene, Encryption, and MultiFactorAuthentication, with RiskAnalysisPolicy using exactMatch and the others closeMatch. Direct Turtle parsing yields 526 triples, 28 classes, 9 object properties, 5 datatype properties, and 31 named individuals.

9.2 Validation Accuracy

Shape-oriented validation was evaluated against the two asserted entity examples:

Entity	Expected SHACL Result	Actual Result	Pass?
Example Compliant Entity	All shapes PASS	All shapes PASS	Yes
Example Non Compliant Entity	Article21Shape FAIL (7)	Article21Shape FAIL (7)	Yes

The structural reasoner reports ExampleCompliantEntity as complete and ExampleNonCompliantEntity as incomplete. It derives five unique standards for the complete entity because several measures share the same standard. This confirms the intended application behavior, while full OWL entailment remains a separate validation task for a standards-complete reasoner.

9.3 Performance Evaluation Scope

The current repository does not contain a dedicated benchmark harness or retained timing dataset. It is therefore not methodologically defensible to report precise average response times as established results. The in-memory architecture is expected to be responsive for a 526-triple graph, but performance claims require a repeatable protocol.

A future benchmark should separate initial RDF parsing from cached requests, record hardware and runtime versions, execute warm-up iterations, collect a sufficiently large sample, report median and percentile latency, and test concurrent clients. Memory consumption and behavior under larger synthetic graphs should also be measured. Until those tests are performed, performance is treated as an implementation characteristic to be evaluated rather than a confirmed thesis result.

9.4 Case Studies

Two asserted entity cases and one user-supplied scenario were examined:

Case Study 1: ExampleCompliantEntity (Essential Entity)

ExampleCompliantEntity implements all twelve Article 21(2) measures. Reasoning output:

classified as CompliantEntity, 100% compliance score, 5 inferred standards (ISO27001 via RiskAnalysisPolicy, ENISAGuidelines via IncidentHandling, ISO27002 via SupplyChainSecurity, NISTFramework via SecureDevelopment, CISControls via BasicCyberHygiene). All three SHACL shapes pass. This case study demonstrates the positive path: a fully compliant essential entity is correctly identified and its standard portfolio is inferred automatically.

Case Study 2: ExampleNonCompliantEntity (Important Entity)

ExampleNonCompliantEntity implements five measures: RiskAnalysisPolicy, IncidentHandling, BasicCyberHygiene, TrainingAwareness, and Encryption. Reasoning output: classified as NonCompliantEntity, 42% compliance score, 7 missing measures (BusinessContinuityManagement, SupplyChainSecurity, SecureDevelopment, EffectivenessAssessment, HumanResourcesSecurity, MultiFactorAuthentication, SecureCommunications). SHACL Article21ComplianceShape reports 7 violations. This case demonstrates compliance gap detection: the seven missing measures represent concrete legal obligations that the entity must address to achieve NIS2 compliance.

Case Study 3: Real-Time Entity Check (MyBank)

A real-time check was performed for "MyBank" as an EssentialEntity implementing all 12 measures. The /api/check-entity endpoint returned: compliant=true, complianceScore=100, inferredClass=CompliantEntity, 5 inferred standards, 4 risk categories addressed. The check was performed entirely in-memory without modifying the ontology, demonstrating the framework's applicability as a compliance support tool for any NIS2-covered organization.

9.5 Comparison with Existing Approaches

The implemented framework is compared against three categories of existing approaches:

Approach	Formal Semantics	Auto Reasoning	SPARQL	SHACL	Open Source
This study (OWL+SHACL)	Yes (OWL 2 DL)	Yes	Yes	Yes	Yes
Commercial GRC tools	No	No	No	No	No
ENISA checklist	No	No	No	No	Yes
Hassan 2025 (Legal Ontology)	Yes (OWL 2 DL)	Yes	Yes	No	Yes
Manual audit	No	No	No	No	N/ A

The comparison shows that the prototype combines several capabilities in one demonstrator. It should not be read as evidence that it is more complete than every commercial or academic alternative, because the compared categories differ in scope, evidence management, workflow, assurance, and production maturity.

9.6 Competency-Question Evaluation

All five competency questions can be expressed using the ontology vocabulary and executed through the query interface. Returned bindings correspond to manually inspected triples and expected inference paths.

This confirms functional adequacy for the declared information needs and demonstrates that the conceptual model supports risk, standard, incident, system, and implementation analysis.

The selected questions use a bounded query subset and do not establish complete SPARQL 1.1 conformance.

9.7 Error Analysis and Negative Cases

The incomplete asserted entity produces seven expected missing-class results rather than a generic failure. Additional partial cases can be generated through /api/check-entity.

Specific reporting supports diagnosis and remediation. The observed SHACL-oriented violations correspond to the absent operational classes in the tested graph.

The small synthetic dataset cannot estimate behavior under noisy data, inconsistent typing, duplicate controls, or disputed mappings.

9.8 Performance and Scalability

The 526-triple graph is small enough for an in-memory prototype, but the repository does not yet contain a repeatable benchmark dataset.

Enterprise deployment would require persistent indexed storage, concurrency testing, query optimization, caching policies, and potentially asynchronous reasoning.

The reported timings should not be extrapolated to large knowledge graphs or production workloads.

9.9 Extended Test Design

The two asserted example entities verify the main complete and incomplete paths, but a stronger evaluation also considers boundaries, invalid input, normalization, and data-quality failures. Table 9.4 defines reproducible cases derived from the implemented service and shape behavior. These cases distinguish ontology-level expectations from API-level request handling.

Case	Input Condition	Expected Result	Purpose
Complete essential entity	12 recognized measure categories	Compliant Entity; 100%; no missing categories	Positive-path classification and standards/ risk enrichment
Incomplete important entity	5 recognized categories	Non Compliant Entity; 42%; seven named gaps	Negative-path gap detection
Boundary entity	11 recognized categories	Non Compliant Entity; 92%; one named gap	All twelve categories are required by the model
Empty measure set	Valid entity metadata; zero categories	Non Compliant Entity; 0%; twelve gaps	Lower boundary and complete omission reporting

Case	Input Condition	Expected Result	Purpose
Duplicate identifiers	Repeated valid category names	Duplicates removed by Set; score based on unique categories	Input normalization
Unknown identifier	Valid categories plus unrecognized value	Unknown value ignored; no score inflation	Vocabulary control
Invalid entity type	Type outside Essential Entity/ Important Entity	HTTP 400 error	Request validation
Missing entity name	Blank or absent name	HTTP 400 error	Mandatory identity field
Missing quality boolean	Measure lacks one Article 21(1) flag	SHACL warning	Quality-property completeness
Invalid risk level	Value outside low/ medium/ high/ critical	SHACL violation	Controlled vocabulary enforcement
Entity typed both categories	Essential Entity and Important Entity	Disjointness violation in reason response	Mutual-exclusion check
Unsupported SPARQL form	Non-SELECT query	HTTP 501 error	Explicit query-engine scope

The cases can be automated as regression tests by starting the service in a controlled environment, submitting fixed requests, and comparing status codes and response fields against stored expectations. Ontology mutations should be tested separately: removing a class, changing a measure type, deleting a quality property, or introducing a conflicting entity type should produce predictable validation changes.

9.10 Interpretation of Coverage Scores

The application calculates complianceScore as the number of unique recognized measure categories divided by twelve, rounded to the nearest whole percentage. This score is a coverage indicator, not a risk-weighted maturity score. Each category contributes equally, although the organizational effort and residual risk associated with the categories can differ substantially. A score of 92 percent therefore means that eleven modeled categories were selected; it does not mean that the entity has achieved 92 percent of legal compliance or 92 percent risk reduction.

Equal weighting is acceptable for communicating the ontology’s minimum-category rule, because the formal CompliantEntity definition requires every category. It would be misleading if reused for prioritization without additional dimensions. A richer assessment could record evidence quality, scope coverage, control effectiveness, risk criticality, remediation age, and confidence. Those dimensions should remain separate from the binary classification so that management can see both whether a category is represented and how strongly it is supported.

9.11 Reproducibility and Independent Verification

A reproducible evaluation should identify the ontology version, application commit, Node.js version, operating environment, input dataset, expected outputs, and timing method. The present report records the ontology IRI and local artifacts but the performance figures should be treated as indicative because the server code does not contain a dedicated benchmark harness. Future measurements should include warm-up, repeated samples, percentile latency, concurrent requests, memory consumption, and separate parsing from query execution.

Independent verification should use a standards-complete RDF and OWL toolchain. The Turtle graph can be parsed and counted independently; the equivalent-class axiom can be tested in Protégé with HermiT or Pellet; the SHACL graph can be executed with pySHACL or TopBraid; and the competency queries can be run in Apache Jena, RDF4J, or another SPARQL 1.1 implementation. Agreement across independent tools would provide stronger evidence than agreement between the ontology and custom code written from the same assumptions.

Chapter 10: Discussion

10.1 Contributions to the Field

This study makes five contributions to the fields of semantic web technology, legal/ regulatory compliance, and cybersecurity governance. First, it provides the first OWL 2 DL ontology for NIS2 Article 21 compliance, filling the gap identified in the literature review between existing OWL compliance ontologies (which address other regulatory domains) and the NIS2 framework. Second, the property chain axiom for usesStandard inference represents a novel application of OWL 2 role chain reasoning to the compliance domain, enabling automatic derivation of standard portfolios from measure implementations without requiring manual usesStandard assertions. Third, a web-based prototype demonstrator integrates bounded OWL-style reasoning, shape-specific structural validation implemented in JavaScript, limited SPARQL SELECT querying, and SKOS alignment in a single inspectable artifact, showing how these complementary mechanisms can coexist in a compliance support tool. Fourth, the on-demand in-memory entity checking endpoint extends the ontology's utility from a static knowledge representation to a structural coverage checker without persisting assessment state. Fifth, the SKOS alignments to ISO 27001, NIST CSF, CIS Controls, and ENISA guidelines establish the ontology as a semantic bridge between NIS2 and the established cybersecurity standards ecosystem.

10.2 Limitations

Several limitations should be acknowledged. First, the reasoning engine is a structural approximation of OWL 2 DL inference rather than a complete description logic reasoner. A full OWL reasoner such as HermiT or Pellet would support additional inference patterns (e.g., property inverses, transitive closure, nominal reasoning) that the custom implementation does not cover. Second, the custom SPARQL engine does not support the full SPARQL 1.1 specification; in particular, UNION, OPTIONAL, MINUS, property paths, and subqueries are not supported. This limits the expressiveness of ad-hoc queries that users can formulate. Third, the ontology models the twelve measures as a flat hierarchy without sub-requirements, whereas the actual Article 21(2) measures involve detailed sub-obligations (e.g., the specific requirements of supply chain security extend to vulnerability handling practices and ICT product security). Fourth, the proportionality assessment (isAppropriate, isProportionate, isStateOfTheArt) is recorded as user-provided boolean values rather than derived from evidence, meaning the ontology cannot automatically assess proportionality from technical evidence. Fifth, the three SHACL shapes declare sh:targetClass :Entity, but no instance is directly typed as :Entity; all instances are typed as :EssentialEntity or :ImportantEntity. A standards-conformant SHACL processor (e.g. pySHACL) resolves this correctly through RDFS class hierarchy inference; the JavaScript implementation compensates by explicitly checking subtype membership. Sixth, the /api/ check-entity endpoint derives inferred standards and addressed risks by looking up example measure instances (named ExampleRiskAnalysisPolicy, ExampleEncryption, etc.) from the ontology; user-submitted measure names that do not correspond to those example individuals will produce empty standard and risk lists, a prototype limitation that

would need to be addressed in a production deployment.

10.3 Future Work

Several directions for future work emerge from this study. First, integration with a production OWL 2 reasoner (HermiT or Pellet via a Java bridge or the owljs library) would provide complete description logic inference, enabling all OWL 2 DL axiom patterns to be processed correctly. Second, extension of the ontology to cover Article 21(3) implementing acts as they are issued by the European Commission would maintain the ontology's regulatory currency. Third, integration with the NIS2 reporting obligations of Article 23 would enable end-to-end compliance management from measure implementation to incident reporting. Fourth, the development of a SPARQL endpoint conformant with the SPARQL 1.1 Protocol specification would enable integration with external tools such as Protégé, RDF4J, or Apache Jena through standard interfaces. Fifth, evaluation with compliance practitioners in actual NIS2-regulated organizations would provide empirical validation of the framework's practical utility and usability beyond the academic evaluation presented here. Sixth, the ontology could be extended to cover the NIS2 governance obligations of Article 20 (management body responsibilities), connecting the technical compliance framework to organizational governance modeling.

10.4 Implications for Compliance Practice

The framework shows how obligations, implemented measures, systems, risks, and standards can be represented in a shared knowledge graph. This can support gap analysis, audit preparation, cross-framework mapping, and regulatory change assessment.

Practical adoption also requires ownership of assertions, evidence-review procedures, version control for mappings, access control, and escalation when automated output conflicts with expert judgment.

Ontology-based assessment is credible only within a governed compliance process with accountable human oversight.

10.5 Generalizability of the Approach

The OWL-SHACL-SPARQL architecture may be adapted to regulations containing identifiable subjects, obligations, controls, and evidence. The transferable contribution is the method rather than the specific Article 21 taxonomy.

Application to DORA, the Cyber Resilience Act, or GDPR would require new legal analysis, competency questions, and validation scenarios. Reusing the current classes unchanged would create superficial alignment.

Generalizability remains a reasoned proposition until the method is evaluated in additional regulatory domains.

10.6 Evidence-Centered Ontology Extension

The most important conceptual extension is an evidence layer. A proposed Assessment individual would evaluate one MeasureImplementation for a defined Entity and scope. It would link to one or more Evidence items, identify an Assessor, record an assessment method and date, and produce a Result such as effective, partially effective, ineffective, or

not assessed. Finding and RemediationAction classes could capture exceptions and improvement work.

This extension separates three claims that are currently compressed: the organization states that a measure exists; evidence shows how it is implemented; and an assessor concludes whether it is adequate. Keeping the claims separate permits disagreement and change over time. A policy owner can assert implementation while an auditor records an exception, without either statement being discarded. Provenance then explains why a summary result changed.

SHACL shapes could require current evidence for high-criticality systems and prevent an expired assessment from supporting a current conclusion. SPARQL could identify measures with no evidence, findings without owners, or critical systems whose latest effectiveness result is negative. OWL could still classify resources and infer relations, while procedural or rule logic would handle time windows and scoring.

10.7 Regulatory Change Management

NIS2 implementation evolves through national transposition, sector guidance, implementing acts, supervisory practice, and updates to referenced standards. A maintainable ontology should distinguish the EU directive provision from jurisdiction-specific obligations and implementation guidance. Each requirement resource should carry jurisdiction, source, effective date, version, and status.

Change impact can then be computed over mappings. If a legal interpretation changes, queries can identify affected shapes, measure classes, evidence requests, systems, and assessment results. Stable identifiers should be retained when meaning remains compatible; breaking semantic changes should receive new version IRIs and migration notes. This is preferable to silently editing labels or constraints in place.

Governance should include a legal reviewer, ontology maintainer, cybersecurity subject-matter expert, and release approver. Proposed changes should include source citations, rationale, affected competency questions, test updates, and backward-compatibility assessment. The ontology becomes a controlled regulatory knowledge product rather than an informal data file.

10.8 Human Oversight and Contestability

Automated compliance results should be contestable. A control owner may challenge a missing classification because evidence was not ingested; an auditor may challenge a positive classification because the asserted measure lacks scope; legal counsel may challenge a mapping because national law differs. The system should preserve these challenges and route them to accountable reviewers.

Human oversight is particularly important for proportionality and state of the art. These concepts depend on organizational context, risk, cost, available technology, and evolving professional practice. A boolean can record a conclusion, but it cannot explain the judgment. The interface should require rationale and evidence and should display who approved the conclusion and when it expires.

Contestability also improves model quality. Repeated disputes may reveal ambiguous classes, missing relationships, or overly strong constraints. Governance data can therefore become feedback for ontology evolution and training material for assessors.

10.9 Production Deployment Architecture

A production architecture would separate the public ontology, organization-specific assessment graph, sensitive evidence repository, reasoning and validation services, and user-facing workflow. The ontology could be stored in version control and published as immutable releases. Assessment data could reside in an authenticated RDF store, while large or sensitive evidence remains in a document repository referenced by controlled identifiers.

Services should use a standards-complete SPARQL endpoint and a tested SHACL processor. Complete OWL reasoning may be performed during ontology release and selected assessment workflows rather than on every request. An API gateway should enforce authentication, authorization, rate limits, and audit logging. Encryption, backup, monitoring, vulnerability management, and incident response apply to the compliance system itself because it contains high-value security information.

Multi-tenancy and separation of duties require particular care. Consultants or group functions may assess several entities, but evidence and findings must remain isolated. Read access to legal vocabulary does not imply access to organizational weaknesses. Roles should distinguish ontology maintainers, evidence contributors, assessors, control owners, executives, and auditors.

10.10 Research Roadmap

Phase	Research Activity	Expected Output	Evaluation
1. Semantic hardening	Run OWL profile checks, complete reasoner tests, SHACL conformance tests	Verified ontology and shape releases	Independent-tool agreement
2. Evidence model	Add assessment, evidence, finding, owner, date, and scope concepts	Evidence-centered ontology module	Expert review and competency questions
3. Legal expansion	Model Article 20, Article 23, implementing acts, and national layers	Versioned NIS2 modules	Legal traceability review
4. Practitioner study	Observe compliance officers using realistic cases	Usability and utility findings	Qualitative study and task metrics
5. Production prototype	Deploy RDF store, identity controls, workflow, and audit logging	Secure multi-user platform	Security and performance testing
6. Cross-regulation study	Map DORA, CRA, GDPR, and sector standards	Interoperability framework	Mapping precision and change-impact evaluation

10.11 Balanced Assessment of the Contribution

The study contribution is strongest as a bounded prototype. It demonstrates how a recent cybersecurity obligation can be decomposed into classes, connected to risks, systems, incidents, and standards, constrained through SHACL, queried through SPARQL patterns,

and exposed through a usable interface. The source artifacts allow the claims to be inspected rather than hidden in a proprietary scoring model.

The contribution is weaker where the current implementation approximates standards or relies on asserted booleans. It does not validate real organizational evidence, execute complete OWL or SHACL semantics, implement full SPARQL 1.1, model national law, or provide practitioner evaluation. Stating these limits directly improves the academic credibility of the work and identifies a concrete program for extension.

Chapter 11: Conclusion

11.1 Summary of Contributions

This study has presented the design, implementation, and evaluation of an OWL ontology for NIS2 Article 21 compliance analysis. The ontology models the ten legal categories through twelve operational classes, using separate classes for the training component of point (g) and for the authentication and communications components of point (j). An equivalent-class axiom represents complete operational coverage, while a property chain derives entity-to-standard relations.

The ontology is embedded in a Node.js/Express demonstrator with graph visualization, structural validation, bounded reasoning, shape-specific checks, limited SPARQL SELECT querying, and real-time entity assessment. Evaluation covered source parsing, structural inventory, two asserted entity cases, competency questions, and an extended test design. Precise performance claims and full standards conformance remain future validation tasks.

11.2 Achievement of Research Objectives

The six objectives from Chapter 1 were addressed at prototype level. RO1 produced an OWL model with twelve operational classes covering the ten legal categories. RO2 implemented bounded category classification and property-chain-equivalent standard derivation. RO3 defined three SHACL shapes and corresponding JavaScript checks. RO4 provided five competency queries over the supported SPARQL subset. RO5 delivered an interactive visualization and entity-checking interface. RO6 added eight SKOS mappings to external cybersecurity resources. Full OWL reasoning, general SHACL execution, complete SPARQL 1.1 support, evidence validation, and practitioner evaluation remain outside the achieved scope.

11.3 Future Research Directions

The work presented in this study establishes a foundation for several further research directions. The most significant near-term extension is integration with a production OWL 2 DL reasoner to replace the structural approximation with complete description logic inference. Medium-term directions include extension to Article 23 incident reporting obligations and Article 20 governance requirements, creating a comprehensive NIS2 compliance ontology. Longer-term research could explore the application of this ontology architecture to other EU cybersecurity regulations (DORA, Cyber Resilience Act) or to the development of a pan-European NIS2 compliance knowledge graph connecting regulated entities, competent authorities, and standards bodies through linked data principles.

The results indicate that formal ontology engineering, grounded in OWL 2 and complementary Semantic Web standards, can support the automation of regulatory compliance verification. As NIS2 obligations come into full force across EU member states, ontology-based compliance tools of the kind presented here have the potential to reduce the manual burden of compliance management, improve the consistency of compliance assessments, and support the development of machine-readable regulatory knowledge that can evolve with the legal framework it represents.

11.4 Final Synthesis of the Research Contribution

The study began with a gap between legal cybersecurity obligations and the machine-processable structures needed for repeatable assessment. It addressed that gap through a bounded formalization of Article 21(2), an explicit compliance-class definition, SHACL constraints, competency queries, and a demonstrable application.

The results support the conclusion that Semantic Web technologies can organize and automate important parts of compliance analysis while retaining traceability to legal categories. The combination of representation, inference, validation, querying, and visualization is more significant than any single component in isolation.

The contribution remains a research prototype. Its responsible use requires recognition of incomplete evidence modeling, limited reasoning coverage, synthetic evaluation data, and the continuing role of legal and cybersecurity expertise.

11.5 Answers to the Research Questions

RQ1: Formal representation of Article 21(2)

Article 21(2) can be represented through a layered ontology that separates regulated entities, risk-management measures, systems, risks, incidents, and standards. The ten legal points are operationalized as twelve classes because the combined requirements in points (g) and (j) are decomposed into separately assessable concepts. Legal provenance is retained through articleReference annotations and explanatory comments.

The representation is sufficiently expressive for category-level coverage and navigation, but not for authoritative compliance determination. Proportionality, effectiveness, evidence quality, national transposition, and temporal validity require additional classes and assessment processes. The answer to RQ1 is therefore affirmative within a clearly bounded conceptual scope.

RQ2: Appropriate OWL axiom patterns

The central pattern is an owl:equivalentClass definition containing Entity and twelve existential restrictions over implementsMeasure. It captures the positive condition that at least one realization of every operational class is present. Inverse properties support bidirectional navigation, a property chain derives entity-to-standard relations, and disjointness documents incompatible high-level categories.

The study also identifies the semantic limits of the pattern. Open-world reasoning does not make an unproved compliant classification equivalent to proved non-compliance. Closed-world gap reporting is therefore implemented by SHACL-oriented checks and procedural comparison. The most appropriate design is not OWL alone, but OWL for meaning and inference combined with explicit validation semantics.

RQ3: Complementarity of SHACL and OWL

SHACL complements OWL by testing submitted graph structure. Qualified minimum counts require one implementation in each operational class; datatype, cardinality, and hasValue constraints assess measure-quality fields; and a controlled list validates risk levels. The resulting messages identify focus nodes and missing categories directly.

OWL remains responsible for class and property semantics, while SHACL expresses completeness expectations. The two mechanisms answer different questions and should not be described interchangeably. Independent execution by a complete SHACL processor remains necessary to validate the custom endpoint against the standard.

RQ4: Competency questions and SPARQL

The ontology vocabulary supports the five competency questions concerning risk coverage, standards, incident prevention, system scope, and entity-measure relations. Their graph patterns can be expressed as SPARQL SELECT queries and checked against the asserted triples. This demonstrates that the model contains the relations needed for the declared analytical tasks.

The local query evaluator supports basic graph patterns and selected filters rather than complete SPARQL 1.1 algebra. RQ4 is answered positively for the included questions, with the explicit qualification that broader query conformance requires a standards-complete engine and test suite.

RQ5: Real-time checking without ontology modification

The /api/check-entity endpoint accepts an entity name, regulatory type, and list of operational classes without writing new triples to the ontology. It normalizes recognized identifiers, calculates unique-class coverage, reports missing classes, and enriches the result with standards and risks derived from the example measure graph.

This proves that the ontology can serve as a stable reference vocabulary for transient assessments. The endpoint remains a self-declaration checker rather than an evidence validator. A production version should persist assessment records separately, connect them to evidence and scope, and maintain provenance and access control.

Research Question	Answer	Principal Evidence	Qualification
RQ1	Yes, at category level	Twelve operational classes covering ten legal points	Evidence and proportionality are not fully modeled
RQ2	Equivalent class plus relational axioms	Restrictions, inverse, disjointness, property chain	Negative classification needs closed-world handling
RQ3	SHACL complements OWL	Coverage, quality, and risk-level constraints	Custom endpoint is not a general SHACL engine
RQ4	Five declared questions are expressible	SPARQL patterns and expected bindings	Local evaluator is a subset
RQ5	Transient checking is feasible	/ api/ check-entity behavior	Submitted claims are not verified evidence

11.6 Claim Boundary and Validation Matrix

Academic conclusions should be proportional to the evidence produced. The matrix below separates directly demonstrated properties from claims that require independent tools, organizational data, or practitioner research. This prevents the existence of an executable prototype from being interpreted as regulatory certification or production readiness.

Claim	Status in This Study	Evidence Available	Further Validation Required
The ontology files are syntactically parseable	Demonstrated	Turtle and RDF/ XML each parse to 526 triples	Continuous parsing in automated tests

Claim	Status in This Study	Evidence Available	Further Validation Required
The legal points are represented	Demonstrated at selected granularity	Ten points mapped to twelve operational classes	Independent legal and sector review
The complete example covers every operational class	Demonstrated	Twelve implements Measure links and class types	Independent reasoner regression test
Missing classes can be reported	Demonstrated procedurally	Seven gaps for the incomplete entity	Broader mutation and boundary tests
The equivalent-class axiom is OWL 2 DL compliant	Plausible but not independently certified here	OWL expression in both serializations	OWL profile validator and complete reasoner
The shapes conform to SHACL semantics	Shape design present; custom checks demonstrated	152-triple shapes graph and endpoint output	py SHACL or equivalent conformance execution
SPARQL 1.1 is fully supported	Not demonstrated	Basic SELECT patterns and limited filters	Standards-complete query engine and test suite
The framework proves legal compliance	Not claimed	Category coverage only	Evidence assessment, legal review, competent authority context
The framework is production scalable	Not demonstrated	Small in-memory graph	Load, concurrency, resilience, and security testing
The interface is usable by practitioners	Not empirically demonstrated	Functional interactive prototype	Structured user study with regulated organizations

11.7 Final Recommendations

First, the ontology should be released from one canonical serialization with automated generation of alternatives. Continuous integration should parse every artifact, compare graph equivalence, execute competency queries, run a complete OWL reasoner, and validate the SHACL graph using an independent processor. This would convert several manually checked assumptions into repeatable quality gates.

Second, the next conceptual increment should be evidence-centered rather than adding more top-level checkboxes. Assessment, Evidence, Finding, Scope, Assessor, ControlOwner, and ReviewDate concepts would allow category assertions to be supported, challenged, and revised. Temporal and provenance information would make results auditable and reduce reliance on unqualified booleans.

Third, practitioner evaluation should precede claims of operational utility. Compliance officers, security architects, auditors, and legal specialists should complete realistic tasks using the framework. Their errors, questions, and disagreements can reveal whether the vocabulary is understandable, whether explanations are sufficient, and whether the model fits actual assessment workflows.

Finally, future legal expansion should remain modular. Article 20 governance, Article 23 incident reporting, implementing acts, and national transposition requirements should be represented in separate but linked modules with explicit version and jurisdiction metadata. This approach preserves the clarity of the Article 21 core while enabling a broader NIS2 compliance knowledge graph.

References

[1] Berners-Lee, T., Hendler, J., & Lassila, O. (2001). The Semantic Web. Scientific American, 284(5), 34–43.

[2] Directive (EU) 2022/2555 of the European Parliament and of the Council of 14 December 2022 on measures for a high common level of cybersecurity across the Union (NIS2 Directive). Official Journal of the European Union, L 333, 80–152.

[3] ENISA (2023). NIS2 Directive: Implementation Guidance. European Union Agency for Cybersecurity. https://www.enisa.europa.eu/topics/nis-directive

[4] Fernández-López, M., Gómez-Pérez, A., & Juristo, N. (1997). METHONTOLOGY: From Ontological Art Towards Ontological Engineering. Proceedings of the AAAI Spring Symposium on Ontological Engineering, 33–40.

[5] Francesconi, E. (2020). Semantic Model for Legal Resources: Annotation and Reasoning Over Legal Texts. Semantic Web, 11(1), 123–135.

[6] Governatori, G., Milosevic, Z., & Sadiq, S. (2006). Compliance Checking Between Business Processes and Business Contracts. Proceedings of the 10th IEEE International EDOC Conference, 221–232.

[7] Gruber, T. R. (1993). A Translation Approach to Portable Ontology Specifications. Knowledge Acquisition, 5(2), 199–220.

[8] Grüninger, M., & Fox, M. S. (1995). Methodology for the Design and Evaluation of Ontologies. Proceedings of IJCAI Workshop on Basic Ontological Issues in Knowledge Sharing.

[9] Harris, S., & Seaborne, A. (Eds.) (2013). SPARQL 1.1 Query Language. W3C Recommendation. https://www.w3.org/TR/sparql11-query/

[10] Hassan, S. B. (2025). Designing a Legal Ontology for Licensing Requirements in Credit Law: A Compliance Checking Case Study with Section 29 of the Australian National Consumer Credit Protection Act 2009. M.Sc. Thesis, University of Florence.

[11] Hitzler, P., Krötzsch, M., Parsia, B., Patel-Schneider, P. F., & Rudolph, S. (Eds.) (2012). OWL 2 Web Ontology Language Primer (2nd ed.). W3C Recommendation. https://www.w3.org/TR/owl2-primer/

[12] ISO/IEC 27001:2022. Information Security, Cybersecurity and Privacy Protection — Information Security Management Systems — Requirements. International Organization for Standardization.

[13] ISO/IEC 27002:2022. Information Security, Cybersecurity and Privacy Protection — Information Security Controls. International Organization for Standardization.

[14] Knublauch, H., & Kontokostas, D. (Eds.) (2017). Shapes Constraint Language (SHACL). W3C Recommendation. https://www.w3.org/TR/shacl/

[15] McGuinness, D. L., & van Harmelen, F. (Eds.) (2004). OWL Web Ontology Language Overview. W3C Recommendation. https://www.w3.org/TR/owl-features/

[16] Miles, A., & Bechhofer, S. (Eds.) (2009). SKOS Simple Knowledge Organization System Reference. W3C Recommendation. https://www.w3.org/TR/skos-reference/

[17] NIST (2018). Framework for Improving Critical Infrastructure Cybersecurity (CSF) v1.1. National Institute of Standards and Technology. https://www.nist.gov/cyberframework

[18] Noy, N. F., & McGuinness, D. L. (2001). Ontology Development 101: A Guide to Creating Your First Ontology. Stanford Knowledge Systems Laboratory Technical Report KSL-01-05.

[19] Palmirani, M., & Governatori, G. (2018). Modelling Legal Knowledge for GDPR Compliance Checking. Proceedings of the Workshop on AI Approaches to the Complexity of Legal Systems, 101–115.

[20] W3C (2012). OWL 2 Web Ontology Language Structural Specification and Functional-Style Syntax (2nd ed.). W3C Recommendation. https://www.w3.org/TR/owl2-syntax/

[21] Berners-Lee, T. (2006). Linked Data. W3C Design Issues. https://www.w3.org/DesignIssues/LinkedData.html

[22] CIS Controls v8 (2021). Center for Internet Security Critical Security Controls. https://www.cisecurity.org/controls/v8